



Machine Learning Summary

Machine learning (Technische Universität München)



Scanne, um auf Studocu zu öffnen

Machine Learning Summary, WS2019

March 25, 2019

Contents

1	Introduction	3
1.1	Supervised Learning	3
1.2	Unsupervised Learning	3
2	Decision Trees	3
2.1	Binary Split	3
2.2	Idea of building optimal DT	3
2.3	Greedy heuristic	3
2.4	How to choose the feature to be split	3
2.5	Build a decision tree using impurity measures	4
2.6	How to split the dataset	4
2.7	K-fold Cross-Validation	4
2.8	Decision tree for regression	4
3	Probabilistic Inference	5
3.1	Maximum Likelihood Estimation (MLE)	5
3.2	Maximum a posteriori Estimation (MAP)	5
3.3	Choose of prior	5
3.4	Fully Bayesian Analysis	6
3.5	How many samples do we need?	7
4	Linear Regression	7
4.1	Regression Problem	7
4.2	Linear Model	8
4.3	Error Function	8
4.4	Optimal Solution	8
4.5	Basis functions	8
4.6	Prevent Overfitting	9
4.7	Bias-Variance tradeoff	9
4.8	Probabilistic Graphical Models	9
4.9	Full Bayesian Approach	11
4.10	Sequential Bayesian Linear Regression	12
5	Linear Classification	12
5.1	Classification vs Regression	12
5.2	Classification Problem	13
5.3	Binary Classification	13
5.4	Multiple Classes Classification	13
5.5	Probabilistic Models for Classification	14
5.6	Generative Model	14
5.7	Discriminative Model	16
5.8	Generative vs Discriminative Model	17
6	Optimization	17
6.1	Convex Set and Convex Function	17
6.2	Convex Optimization	19
6.3	Gradient Descent	19
6.4	Other Optimization Approaches	20
6.5	Newton Method	21

7	Constrained Optimization	22
7.1	Inequality Constraints	22
7.2	Lagrangian	22
7.3	Duality and Recipe for solving COP	23
7.4	KKT condition	23
7.5	Projected Gradient Descent	24
8	SVM	24
8.1	Hyperplane	24
8.2	Optimization Problem	25
8.3	Soft Margin SVM	27
9	Kernels	29
9.1	Feature space	29
9.2	Kernel trick	29
9.3	Kernelized SVM	30
10	Deep Learning	30
10.1	Feed-Forward Neural Network	30
10.2	Activation functions	31
10.3	Choice of loss and last layer output	31
10.4	Parameter Learning	32
10.5	CNN	33
10.6	RNN	33
10.7	Training deep neural network	33
11	PCA	34
11.1	Determin the principle component	34
11.2	Dimension reduction with PCA	35
11.3	Alternative views of PCA	36
11.4	PPCA	36
12	SVD	38
12.1	Definition	38
12.2	Best approximation	38
12.3	SVD and PCA: Comparison	38
13	Matrix Factorization	39
13.1	Latent Factor Model	39
13.2	Alternating Optimization	39
13.3	Rating Prediction	40
13.4	L2 vs. L1 Regularization	40
13.5	Further Factorization Models	41
13.6	Autoencoder	41

1 Introduction

1.1 Supervised Learning

Given training samples: $\mathbf{X}_{train} = \{x_1, \dots, x_N\}$ with corresponding **targets** $\mathbf{y}_{train} = \{y_1, \dots, y_N\}$
Find a function f that generalizes this relationship $f(x_i) \approx y_i$ and use it for prediction on the test data $\mathbf{X}_{test}$.

Classification: If the targets y_i represent categories, the problem is called *classification*

Regression: If the targets y_i represent continuous numbers, the problem is called regression

1.2 Unsupervised Learning

concerned with finding structure in unlabeled data.

Clustering: group similar objects together

Dimensionality reduction: project down high-dimensional data

Generative modeling: generate new realistic data

2 Decision Trees

2.1 Binary Split

Split on a single feature $x_i \leq a$. Leaf nodes represent the distribution of classes and can't be further split.

2.2 Idea of building optimal DT

Optimal means that DT performs the best on unseen data.

Instead: we grow the tree top-down and choose the best split node-by-node using a greedy heuristic on training data.

2.3 Greedy heuristic

Missclassification rate:

$$i_E(t) = 1 - \max_c p(y = c|t) = 1 - \max_c \pi_c$$

with t denotes the node which is currently split, means that we treat the classes which are not dominant as misclassified, so the missclassification rate is to subtract the portion of dominant class from 1.

We split the node if it improves the misclassification rate, which means $\Delta i(s, t) > 0$.

The Improvement of the missclassification rate:

$$\Delta i(s, t) = i(t) - p_L \cdot i(t_L) - p_R \cdot i(t_R)$$

with s denotes the split operation and t_L the left subtree and t_R the right subtree after split. p_L and p_R are the portion of the dominance classes.

2.4 How to choose the feature to be split

The number of possible splits = number of features $\times$ number of attributes. A feature could have many attributes. We use greedy heuristic on the training set to best split the tree node-by-node, the split should therefore minimize the impurity of the split region.

For impurity measures of the split data, we usually use missclassification rate (linear), entropy (non-linear), gini-index (non-linear) etc.

Entropy

$$i_H(t) = - \sum_{c_i \in C} \pi_{c_i} \log \pi_{c_i}$$

Gini-index

$$i_G(t) = \sum_{c_i \in C} \pi_{c_i}(1 - \pi_{c_i}) = 1 - \sum_{c_i \in C} \pi_{c_i}^2$$

π_{c_i} denotes the portion of dominance of class c_i , and $1 - \pi_{c_i}$ denotes the probability that the class c_i is misclassified as other classes.

Shannon Entropy For a discrete random variable X with possible values $\{x_1, \dots, x_n\}$

$$\mathbb{H}(X) = - \sum_i^n p(X = x_i) \log_2 p(X = x_i)$$

Higher entropy $\rightarrow$ higher impurity, lower entropy $\rightarrow$ lower impurity.

2.5 Build a decision tree using impurity measures

Step 1: compute the Gini-index $i_G(t)$ for the root node t .

Step 2: compare all possible splits and choose the split s with **largest** Gini-index **gain** $\Delta i_G(s, t)$. The larger the information gain is, the purer is the result after the split.

Step 3: iterate till the stop criterias are fulfilled. $\rightarrow$ this leads to the overfitting problem, which means the decision tree has poor generalization. This can be observed by low training error and high validation error, or the point when the validation loss starts to head up.

Stopping criteria for growing the tree

- the distribution is pure, i.e. $i_G(t) = 0$.
- maximum depth reached.
- number of samples in each branch below certain threshold.
- information gain $\Delta i_G(s, t)$ is below certain threshold.
- accuracy on the validation set is good enough.
- or we can try to overfit the tree first, then do pruning

2.6 How to split the dataset

Split the entire dataset into **Learning set** and **Test set**, then split the Learning set into **Training set** and **Validation set**. Use the training set to build the trees with different parameters, then use the validation set to pick up the tree which performs the best by comparing the predictions to the true labels, then report the final performance on the test set. Only touch the test set for one time at the end!

2.7 K-fold Cross-Validation

If the dataset is relatively small, we might wanna split the learning set multiple times in different ways, in order to have an overall estimation of the model.

- Split the learning set into K folds ($K = 10$ is commonly used).
- Use $K - 1$ folds for training and the remaining one for validation, train K models on each split of learning set.
- Use the average performance of K models to set up hyperparameters.
- Use all the learning set and the best parameter to train a final model and then report the final performance from it.

2.8 Decision tree for regression

For regression the y is real value rather than a class. At the leaves we compute the mean over the outputs and we use mean-squared-error as splitting heuristic.

3 Probabilistic Inference

3.1 Maximum Likelihood Estimation (MLE)

Step 1: write down likelihood function, take logarithm (monotonic functions preserve the maximum position)
Step 2: take derivative and set it to zero and obtain the θ which maximizes the log likelihood

Example Given a coin flip sequence which is H,T,H,H,T,H,H,H,T,H and denote $p(F_i = T) = \theta$ with i.i.d assumption, thus we have

$$p(HTHHTHHHTH|\theta) = \theta^3(1-\theta)^7$$

Take logarithm and compute the derivative w.r.t θ we get

$$\begin{aligned}\theta_{MLE} &\Leftarrow \frac{d}{d\theta} \theta^3(1-\theta)^7 \stackrel{!}{=} 0 \\ &\Leftrightarrow \frac{d}{d\theta} \log(\theta^3(1-\theta)^7) = 0 \\ &\Leftrightarrow \frac{d}{d\theta} 3\log(\theta) + 7\log(1-\theta) = 0 \\ &\Leftrightarrow 3\frac{1}{\theta} - 7\frac{1}{1-\theta} = 0 \\ &\Leftrightarrow 3(1-\theta) - 7\theta = 0 \\ &\Leftrightarrow 3 - 3\theta - 7\theta = 0 \\ &\Leftrightarrow \theta = \frac{3}{10}\end{aligned}$$

MLE for any coin sequence: $\theta_{MLE} = \frac{|T|}{|T|+|H|}$, if we want to find the probability of next coin flip is T .

3.2 Maximum a posteriori Estimation (MAP)

Bayes rule:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}$$

posterior $\propto$ likelihood $\cdot$ prior

Remember posterior and prior are estimations for model parameter θ and likelihood is estimation for data $\mathcal{D}$.

MLE corresponds to having uniform prior, means that we don't have any informative knowledge about the distribution of model parameter θ .

3.3 Choose of prior

Never choose zero prior, otherwise no matter what likelihood we have, we always get zero posterior.

A common way of choosing prior is that we use **conjugate priors**, i.e. the prior which has similar form as the likelihood to make the computation easier. If we have Bernoulli likelihood, we can choose the conjugate prior of Bernoulli distribution, i.e. the Beta distribution:

$$\text{Beta}(\theta|a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1-\theta)^{b-1}, \quad \theta \in [0, 1]$$

where $\Gamma(n) = (n-1)!$, if $n \in \mathbb{N}$. For $a = b = 1$, Beta distribution is same as uniform distribution.

Example Given a coin flip sequence with $|T|$ and $|H|$ denote the the number of Tails and Heads repectively, and we set $p(F_i = T) = \theta$, so we have the likelihood $p(\mathcal{D}|\theta) = \theta^{|T|}(1-\theta)^{|H|}$. Choose the conjugate prior as

Beta($\theta|a, b$), we can then compute the posterior through the Bayes rule

$$\begin{aligned}
p(\theta|\mathcal{D}) &= \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})} \\
&= \frac{\frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)}\theta^{a-1}(1-\theta)^{b-1}\theta^{|\mathcal{T}|}(1-\theta)^{|\mathcal{H}|}}{p(\mathcal{D})} \\
&= \underbrace{\frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)}\frac{1}{p(\mathcal{D})}}_{\text{constant w.r.t } \theta} \theta^{a-1+|\mathcal{T}|}(1-\theta)^{b-1+|\mathcal{H}|}
\end{aligned}$$

We know that $\int cf(x)dx = 1 \Rightarrow \int f(x)dx = \frac{1}{c}$ if $f(x)$ is probability distribution. Since the posterior is also a Beta distribution, thus $\int p(\theta|\mathcal{D})d\theta = 1$, so the constant part must have the form which makes the integral up to 1, thus we need to fit the constant part in form of Beta distribution and get

$$\frac{\Gamma(|\mathcal{H}| + |\mathcal{T}| + a + b)}{\Gamma(|\mathcal{T}| + a)\Gamma(|\mathcal{H}| + b)}$$

Thus we have $p(\theta|\mathcal{D}) = \text{Beta}(\theta|a + |\mathcal{T}|, b + |\mathcal{H}|)$.

Same as MLE, take derivative w.r.t θ and set it to zero, we then obtain the MAP estimation for θ

$$\theta_{MAP} = \frac{|\mathcal{T}| + a - 1}{|\mathcal{H}| + |\mathcal{T}| + a + b - 2}$$

If we set $p(F_i = H) = \theta$, we then have MAP estimation for θ

$$\theta_{MAP} = \frac{|\mathcal{H}| + a - 1}{|\mathcal{H}| + |\mathcal{T}| + a + b - 2}$$

The **mode** of Beta(a, b) is $\frac{a-1}{a+b-2}$, for $a, b > 1$, thus we see that $\theta_{MAP} = \frac{|\mathcal{T}|+a-1}{|\mathcal{H}|+|\mathcal{T}|+a+b-2}$ is the mode of the posterior distribution, means that if we know the distribution of posterior probability, we can easily compute the MAP estimation. And if we use the uniform prior, e.g. Beta(a, b) for $a = b = 1$

$$\theta_{MAP} = \frac{|\mathcal{T}| + 1 - 1}{|\mathcal{H}| + |\mathcal{T}| + a + b - 2} = \frac{|\mathcal{T}|}{|\mathcal{H}| + |\mathcal{T}|} \Rightarrow \theta_{MLE}$$

we'll obtain the MLE estimation in the end.

3.4 Fully Bayesian Analysis

Posterior Predictive Distribution We wanna predict the probability that the next coin flip is T , given observations $\mathcal{D}$ and prior parameters a, b , so we wanna know $p(F = T|\mathcal{D}, a, b)$, also called posterior predictive distribution. Since a flip is dependend on θ , but we don't know what θ is, i.e. we don't know the probability of Tails in this case, so we integrate over all possible values of θ

$$p(F = T|\mathcal{D}, a, b) = \int_0^1 p(F = T, \theta|\mathcal{D}, a, b)d\theta$$

means that we take all possible θ into consideration, thus the posterior predictive distribution is **not** point estimation anymore, but an estimation over all possible θ . This is so called **fully Bayesian analysis**.

Use product rule and denote $f = 0$ for Heads and $f = 1$ for Tails we get

$$\begin{aligned}
p(F = f|\mathcal{D}, a, b) &= \int_0^1 p(F = f, \theta|\mathcal{D}, a, b) d\theta \\
&= \int_0^1 p(f, \theta|\mathcal{D}, a, b) d\theta \\
&= \int_0^1 \underbrace{p(f|\theta, \mathcal{D}, a, b)}_{\text{cond. ind.}} p(\theta|\mathcal{D}, a, b) d\theta \\
&= \int_0^1 p(f|\theta) p(\theta|\mathcal{D}, a, b) d\theta \\
&= \int_0^1 \theta^f (1-\theta)^{1-f} \frac{\Gamma(|T| + a + |H| + b)}{\Gamma(|T| + a) \Gamma(|H| + b)} \theta^{|T|+a-1} \\
&\quad (1-\theta)^{|H|+b-1} d\theta \\
&= \frac{\Gamma(|T| + a + |H| + b)}{\Gamma(|T| + a) \Gamma(|H| + b)} \int_0^1 \theta^{f+|T|+a-1} (1-\theta)^{-f+|H|+b} d\theta \\
&= \frac{\Gamma(|T| + a + |H| + b)}{\Gamma(|T| + a) \Gamma(|H| + b)} \frac{\Gamma(f + |T| + a) \Gamma(|H| + b - f + 1)}{\Gamma(|T| + a + |H| + b + 1)} \\
&= \frac{(f + |T| + a - 1)! (|H| + b - f)!}{(|T| + a - 1)! (|H| + b - 1)!} \frac{1}{|T| + a + |H| + b}
\end{aligned}$$

Thus we have

$$\begin{aligned}
p(F = f|\mathcal{D}, a, b) &= \frac{1}{|T| + a + |H| + b} \cdot \begin{cases} |H| + b, & f = 0 \\ |T| + a, & f = 1 \end{cases} \\
&= \frac{(|T| + a)^f (|H| + b)^{1-f}}{|T| + a + |H| + b}
\end{aligned}$$

When a lot of data are available, the influence of prior drops, i.e. the posterior is dominated by the data. With more data the posterior becomes more peaky, means that we are more certain about our estimate of θ . Remember: θ_{MLE} and θ_{MAP} are point estimations, while θ_{FB} is an estimation over all possible θ .

3.5 How many damples do we need?

Hoeffding's Inequality For a a sampling complexity bound we have

$$p(|\theta_{MLE} - \theta| \geq \epsilon) \leq 2e^{-2N\epsilon^2} \leq \delta$$

with $N = |T| + |H|$ number of samples and $1 - \delta$ the probability of confidence. This says that if we want the θ_{MLE} deviates from the true value more than ϵ with probability at least $1 - \delta$, we need at least

$$N \geq \frac{\ln(2/\delta)}{2\epsilon^2}$$

many samples.

4 Linear Regression

4.1 Regression Problem

Given observations (input) $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$, $\mathbf{x}_i \in \mathbb{R}^D$ (each input is a D -dimensional feature vector) and targets (output) $\mathbf{y} = (y_1, \dots, y_N)$, $y_i \in \mathbb{R}$ (targets of linear regression are **real numbers!**), the task of linear regression is to find a mapping $f(\cdot)$ from inputs to outputs so that

$$\begin{aligned}
y_i &\approx f(\mathbf{x}_i) \\
\Rightarrow y_i &= f(\mathbf{x}_i) + \underbrace{\epsilon_i}_{\text{noise}}, \quad \epsilon_i \sim \mathbf{N}(0, \beta^{-1})
\end{aligned}$$

The noise has zero-mean Gaussian distribution with a fixed precision $\beta = \frac{1}{\sigma^2}$, thus we know that the target also has Gaussian distribution with mean of $f_{\mathbf{w}}(\mathbf{x}_i)$ and precision β^{-1} , i.e. $y_i \sim \mathcal{N}(f_{\mathbf{w}}(\mathbf{x}_i), \beta^{-1})$.

4.2 Linear Model

We call it a linear model of $\mathbf{x}_i$ if we choose $f(\mathbf{x})$ to be a linear function $f_w(\mathbf{x}_i) = w_0 + w_1x_{i1} + w_2x_{i2} + \dots + w_Dx_{iD}$ for x_{iN} the N-th dimension of the vector $\mathbf{x}_i$, $N = 1, \dots, D$

We can absorb the bias term w_0 into $\tilde{\mathbf{w}} = (w_0, \dots, w_N)$ and add 1 to the feature vector $\mathbf{x}$ to have $\tilde{\mathbf{x}} = (1, \mathbf{x}_1, \dots, \mathbf{x}_N)$, so that $f_{\mathbf{w}}(\tilde{\mathbf{x}}) = \tilde{\mathbf{w}}^T \tilde{\mathbf{x}}$

4.3 Error Function

Error function gives a measure of misfit between the model and observed data.

Least-Square Error

$$\begin{aligned} E_{LS}(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^N (f_{\mathbf{w}}(\mathbf{x}_i) - y_i)^2 \\ &= \frac{1}{2} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}_i - y_i)^2 \end{aligned}$$

4.4 Optimal Solution

Find the optimal weight vector $\mathbf{w}^*$ that minimizes the error

$$\begin{aligned} \mathbf{w}^* &= \arg \min_{\mathbf{w}} E_{LS}(\mathbf{w}) \\ &= \arg \min_{\mathbf{w}} \frac{1}{2} \sum_{i=1}^N (\mathbf{w}^T \mathbf{x}_i - y_i)^2 \end{aligned}$$

Denote the observations $\mathbf{x}_i$ as rows of the matrix $\mathbf{X} \in \mathbb{R}^{N \times D}$ and write in matrix expression, we get

$$\mathbf{w}^* = \arg \min_{\mathbf{w}} \frac{1}{2} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y})$$

Compute the gradient we get

$$\begin{aligned} \nabla_{\mathbf{w}} E_{LS}(\mathbf{w}) &= \nabla_{\mathbf{w}} \frac{1}{2} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ &= \nabla_{\mathbf{w}} \frac{1}{2} (\mathbf{w}^T \mathbf{X}^T \mathbf{X} \mathbf{w} - 2\mathbf{w}^T \mathbf{X}^T \mathbf{y} + \mathbf{y}^T \mathbf{y}) \\ &= \mathbf{X}^T \mathbf{X} \mathbf{w} - \mathbf{X}^T \mathbf{y} \end{aligned}$$

Set the gradient to zero to have close-form optimal weight

$$\mathbf{w}^* = \underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{=\mathbf{X}^\dagger} \mathbf{y}$$

with $\mathbf{X}^\dagger$ the Moore-Penrose pseudo-inverse of $\mathbf{X}$.

4.5 Basis functions

Use basis function to do some feature engineering, not linear in $\mathbf{x}$ anymore but still linear in $\mathbf{w}$

$$f_{\mathbf{w}}(x) = w_0 + \sum_{j=1}^M w_j \phi_j(x) = \mathbf{w}^T \phi(x)$$

Polynomials $\phi_j(x) = x^j$

If we choose a very large M for degree of the polynomial, we might get the overfitting problem. To choose suitable M , we can use the standard train-validation split approach. Overfitting appears when:

1. the validation error starts to head up while training error drops consistently.
2. the model parameter $\mathbf{w}$ becomes very large

Gaussian $\phi_j(x) = e^{\frac{-(x-\mu_j)^2}{2s^2}}$

Sigmoid $\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right)$

By using basis function, the error function becomes

$$\begin{aligned} E_{LS}(\mathbf{w}) &= \frac{1}{2} \sum_{i=1}^N (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 \\ &= \frac{1}{2} (\Phi \mathbf{w} - \mathbf{y})^T (\Phi \mathbf{w} - \mathbf{y}) \end{aligned}$$

where $\Phi \in \mathbb{R}^{N \times (M)}$ is the design matrix of ϕ and has the form

$$\Phi = \begin{pmatrix} \phi_0(x_1) & \phi_1(x_1) & \dots & \phi_{M-1}(x_1) \\ \phi_0(x_2) & \phi_1(x_2) & & \vdots \\ \vdots & \vdots & \ddots & \\ \phi_0(x_N) & \phi_1(x_N) & \dots & \phi_{M-1}(x_N) \end{pmatrix}$$

Again, compute the gradient and set it to zero, we get the close-form optimal weight

$$\mathbf{w}^* = \underbrace{(\Phi^T \Phi)^{-1} \Phi^T}_{\Phi^\dagger} \mathbf{y}$$

Compare to $\mathbf{w}^* = \underbrace{(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T}_{=\mathbf{X}^\dagger} \mathbf{y}$ we see that the least-square is just doing a projection from the space

spanned by basis function.

4.6 Provent Overfitting

To prevent overfitting, we can add regularization term to the error function for penalizing large weights. We call least-square error function with L2 regularization the **ridge regression**.

$$E_{ridge}(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^N (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2$$

If we choose a too large λ ($\lambda \rightarrow \infty$), the model will not fit the data and we end up having all weights closed to zero, i.e. larger regularization strength λ leads to smaller weights $\mathbf{w}$. If we choose a too small λ , the regularization will be weak and thus can't prevent the overfitting issue.

4.7 Bias-Variance tradeoff

Bias: the average prediction over **all** data sets differs from the desired regression function

Variance: the extent to which the solutions for **individual** data sets vary around their average

High bias: the model is too **rigid** to fit the underlying data distribution (the model is bad). This could happen if the model is misspecified and/or the regularization strength λ is too high.

High variance: the model is too **flexibel** and therefore captures noise in the data (the model is too powerful for the data), i.e. overfitting. This could happen if the model fits the underlying data too well and/or the regularization strength λ is too small.

Tradeoff: select a model with large capacity (a model that fits the data well enough) and keep the variance under control by choosing appropriate regularization strength λ .

4.8 Probabilistic Graphical Models

We now try to explain the linear regression from a probabilistic view, we try to compute the maximum likelihood estimation and maximum a posteriori estimations for model parameters, i.e. we want to compute $\mathbf{w}_{ML}, \beta_{ML}$ and $\mathbf{w}_{MAP}, \beta_{MAP}$ from probabilistic perspective.

Bayesian Network A way of representing **joint** distribution via a directed acyclic graph.

$$p(x_1, \dots, x_{|V|}) = \prod_{n=1}^{|V|} p(x_n | Pa(x_n))$$

where $Pa(x_n)$ is the set containing the parent nodes of x_n in the bayesian network. To decompose the joint probability from bayesian network, we need to obtain the dependencies of nodes from the bayesian network. We can define a set of parameters $\{\theta_1, \dots, \theta_{|V|}\}$ to parameterize the joint distribution, i.e. θ_n parameterizes the factor $p(x_n|Pa(x_n))$.

In general if we have n random variables, we need then 2^n parameters to describe the joint probability distribution over n random variables.

Recall that the task of linear regression is to find a mapping f from observations to targets under the assumption that the noise has a zero-mean Gaussian distribution

$$y_i = f_{\mathbf{w}}(\mathbf{x}_i) + \underbrace{\epsilon_i}_{\text{noise}}, \quad \epsilon \sim \mathcal{N}(0, \beta^{-1})$$

that is, the targets depend on the model parameters $\mathbf{w}, \beta$ and the observations $x_1, \dots, x_n$, so we can construct a bayesian network to describe the dependencies and compute the likelihood of a single sample $p(y_i|f_{\mathbf{w}}(\mathbf{x}_i), \beta) = \mathcal{N}(y_i|f_{\mathbf{w}}(\mathbf{x}_i), \beta)$, so the likelihood of entire dataset $\mathcal{D} = \{\mathbf{X}, \mathbf{y}\}$ is

$$p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta) = \prod_{i=1}^N p(y_i|f_{\mathbf{w}}(\mathbf{x}_i), \beta)$$

Since we have the likelihood of targets given the data and model parameters, we can use maximum likelihood to compute the model parameters $\mathbf{w}, \beta$ which maximize the likelihood function

$$\begin{aligned} \mathbf{w}_{ML}, \beta_{ML} &= \arg \max_{\mathbf{w}, \beta} p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta) \\ &= \arg \max_{\mathbf{w}, \beta} \ln p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta) \\ &= \arg \min_{\mathbf{w}, \beta} -\ln p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta) \\ &= \arg \min_{\mathbf{w}, \beta} E_{ML}(\mathbf{w}, \beta) \end{aligned}$$

with $E_{ML}(\mathbf{w}, \beta)$ the maximum likelihood error function

$$\begin{aligned} E_{ML}(\mathbf{w}, \beta) &= -\ln p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta) \\ &= -\ln \left[\prod_{i=1}^N \mathcal{N}(y_i|f_{\mathbf{w}}(\mathbf{x}_i), \beta^{-1}) \right] \\ &= -\ln \left[\prod_{i=1}^N \sqrt{\frac{\beta}{2\pi}} \exp \left(-\frac{(\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2}{2\beta^{-1}} \right) \right] \\ &= -\sum_{i=1}^N \ln \left[\sqrt{\frac{\beta}{2\pi}} \exp \left(-\frac{(\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2}{2\beta^{-1}} \right) \right] \\ &= -\sum_{i=1}^N \left[\ln \sqrt{\frac{\beta}{2\pi}} + \ln \exp \left(-\frac{\beta}{2} (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 \right) \right] \\ &= -\sum_{i=1}^N \frac{1}{2} \ln \frac{\beta}{2\pi} + \sum_{i=1}^N \frac{\beta}{2} (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 \\ &= -\sum_{i=1}^N \frac{1}{2} (\ln \beta - \ln 2\pi) + \sum_{i=1}^N \frac{\beta}{2} (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 \\ &= -\frac{N}{2} \ln \beta + \frac{N}{2} \ln 2\pi + \sum_{i=1}^N \frac{\beta}{2} (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 \end{aligned}$$

Compute the derivatives w.r.t $\mathbf{w}$ and β and set them to zero (for β_{ML} we need to plug in $\mathbf{w}_{ML}$ into derivative), we obtain the optimal model parameter $\mathbf{w}_{ML}$ and β_{ML}

$$\begin{aligned} \mathbf{w}_{ML} &= (\Phi^T \Phi)^{-1} \Phi^T \mathbf{y} = \Phi^\dagger \mathbf{y} \\ \beta_{ML} &= \frac{1}{N} \sum_{i=1}^N (\mathbf{w}_{ML}^T \phi(\mathbf{x}_i) - y_i)^2 \end{aligned}$$

We see that maximizing the likelihood is equivalent to minimizing the least-squares error function.

Also, plugging in the $\mathbf{w}_{ML}$ and β_{ML} into the likelihood we get a **predictive distribution** which can be used to make prediction $\tilde{y}_{new}$ for new data $\mathbf{x}_{new}$

$$p(\tilde{y}_{new}|\mathbf{x}_{new}, \mathbf{w}_{ML}, \beta_{ML}) = \mathcal{N}(\tilde{y}_{new}|\mathbf{w}_{ML}^T \phi(\mathbf{x}_{new}), \beta_{ML}^{-1})$$

Since we have likelihood, we can also compute the posterior distribution by introducing some dependencies for weights $\mathbf{w}$

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \beta, \alpha) = \frac{\overbrace{p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta)}^{\text{likelihood}} \cdot \overbrace{p(\mathbf{w}|\alpha)}^{\text{prior}}}{\underbrace{p(\mathbf{X}, \mathbf{y})}_{\text{normalizing constant}}}$$

We can choose the prior to be an isotropic multivariate normal distribution with zero mean

$$p(\mathbf{w}|\alpha) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \alpha^{-1}\mathbf{I}) = \left(\frac{\alpha}{2\pi}\right)^{\frac{M}{2}} \exp\left(-\frac{\alpha}{2}\mathbf{w}^T \mathbf{w}\right)$$

where α the precision of the distribution and M the number of elements in the vector w .

Find the $\mathbf{w}$ which maximizes the posteriori, define the MAP error function as

$$\begin{aligned} E_{MAP}(\mathbf{w}) &= \ln p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \alpha, \beta) \\ &= \ln p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta) + \ln p(\mathbf{w}|\alpha) - \underbrace{\ln p(\mathbf{X}, \mathbf{y})}_{\text{constant}} \\ &= -\ln p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta) - \ln p(\mathbf{w}|\alpha) + \text{constant} \\ &= -\frac{N}{2} \ln \beta + \frac{N}{2} \ln 2\pi + \sum_{i=1}^N \frac{\beta}{2} (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 \\ &\quad - \frac{M}{2} \ln \frac{\alpha}{2} + \frac{\alpha}{2} \mathbf{w}^T \mathbf{w} \\ &= \frac{\beta}{2} \sum_{i=1}^N (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 + \frac{\alpha}{2} \|\mathbf{w}\|_2^2 + \text{constant} \\ &= \underbrace{\frac{1}{2} \sum_{i=1}^N (\mathbf{w}^T \phi(\mathbf{x}_i) - y_i)^2 + \frac{\lambda}{2} \|\mathbf{w}\|_2^2}_{\text{ridge regression error function!}} + \text{constant} \end{aligned}$$

where $\lambda = \frac{\alpha}{\beta}$. We see that MAP estimation with Gaussian prior is equivalent to ridge regression.

Same as maximum likelihood, we can plug in $\mathbf{w}_{MAP}$ to get predictive distribution which can be used to predict new data $\mathbf{x}_{new}$

$$p(\tilde{y}_{new}|\mathbf{x}_{new}, \mathbf{w}_{MAP}, \beta) = \mathcal{N}(\tilde{y}_{new}|\mathbf{w}_{new}^T \phi(\mathbf{x}_{new}), \beta^{-1})$$

To find $\mathbf{w}_{MAP}$, we minimize the MAP error function as usual

$$\mathbf{w}_{MAP} = \arg \min_{\mathbf{w}} E_{MAP}(\mathbf{w})$$

4.9 Full Bayesian Approach

Full bayesian approach enables us to estimate the full posterior distribution $p(\mathbf{w}|\mathcal{D})$, means that we don't limit ourselves to $\mathbf{w}_{MAP}$ which is a point estimation.

$$p(\mathbf{w}|\mathcal{D}) \propto \overbrace{p(\mathbf{y}|\mathbf{X}, \mathbf{w}, \beta)}^{\text{likelihood}} \overbrace{p(\mathbf{w}|\alpha)}^{\text{prior}}$$

Since both likelihood and prior are Gaussian, we have a close-form posterior (due to the fact that Gaussian is its own conjugate prior)

$$p(\mathbf{w}|\mathbf{X}, \mathbf{y}, \alpha, \beta) = \mathcal{N}(\mathbf{w}|\mathbf{m}_N, \mathbf{S}_N)$$

with

$$\begin{aligned} \mathbf{m}_N &= \mathbf{S}_N (\mathbf{S}_0^{-1} \mathbf{m}_0 + \beta \Phi^T \mathbf{y}) \\ \mathbf{S}_N^{-1} &= \mathbf{S}_0^{-1} + \beta \Phi^T \Phi \end{aligned}$$

where $\mathbf{m}_0$ and $\mathbf{S}_0$ the prior mean and prior covariance.

Properties of the Posterior Distribution

- posterior is a gaussian, therefore max. post. solution equals the mode: $\mathbf{w}_{MAP} = \mathbf{m}_N$
- infinitely broad prior $\mathbf{S}_0^{-1} \rightarrow 0$ therefore $\mathbf{w}_{MAP} \rightarrow \mathbf{w}_{ML} = \Phi^\dagger \mathbf{y}$
- for $N = 0$ data points, we get back prior

Posterior Distribution with simple prior

$$p(\mathbf{w}|\alpha) = \mathcal{N}(\mathbf{w}|\mathbf{m}_0 = 0, \mathbf{S}_0 = \alpha^{-1}\mathbf{I})$$

$$\mathbf{m}_N = \beta \mathbf{S}_N \Phi^T \mathbf{y}$$

$$\mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^T \Phi$$

4.10 Sequential Bayesian Linear Regression

Use this approach when:

- the dataset is too large and can't be imported into memory all at once
- data arrives sequentially (in a stream) and has to be processed in an online manner

The solution is, we iteratively use the posterior from previous time step as the prior at next time step, until all data are processed.

1. Process the first batch of data $\mathcal{D}_1$ at time step $t = 1$ and obtain the posterior

$$p(\mathbf{w}|\mathcal{D}_1) \propto p(\mathcal{D}_1|\mathbf{w})p(\mathbf{w}|\alpha)$$

2. Use the posterior from step $t = 1$ as prior for step $t = 2$

$$\begin{aligned} p(\mathbf{w}|\mathcal{D}_2, \mathcal{D}_1) &\propto p(\mathcal{D}_2|\mathbf{w})p(\mathcal{D}_1|\mathbf{w})p(\mathbf{w}|\alpha) \\ &\propto p(\mathcal{D}_2|\mathbf{w})p(\mathbf{w}|\mathcal{D}_1) \end{aligned}$$

3. Iterate until all data $\mathcal{D}$ are processed.

A simple example Bayesian regression for the target values

$$y_i = -0.3 + 0.5x_i + \epsilon, \quad \epsilon \sim \mathcal{N}(0, 0.2^2)$$

We set the basis function

$$\phi(x) = \begin{bmatrix} 1 \\ x \end{bmatrix}$$

thus we have

$$f_{\mathbf{w}}(x) = w_0 + w_1 x$$

Need to notice: outliers disturb the (correct) estimation

Predictive Distribution Integration over $\mathbf{w}$ of likelihood and posterior. Usually we want to predict output $\tilde{y}_{new}$ for new values of $\mathbf{x}_{new}$

$$\begin{aligned} p(\tilde{y}_{new}|\mathbf{x}_{new}, \mathcal{D}) &= \int \underbrace{p(\tilde{y}_{new}, \mathbf{w}|\mathbf{x}_{new}, \mathcal{D})}_{\text{likelihood}} \underbrace{p(\mathbf{w}|\mathcal{D})}_{\text{posterior}} d\mathbf{w} \\ &\Rightarrow \mathcal{N}(\tilde{y}_{new}|\mathbf{m}_N^T \phi(\mathbf{x}_{new}), \beta^{-1} + \phi(\mathbf{x}_{new})^T \mathbf{S}_N \phi(\mathbf{x}_{new})) \end{aligned}$$

5 Linear Classification

5.1 Classification vs Regression

Regression The output y of regression problem is continuous, i.e. $y \in \mathbb{R}$

Classification The output y of classification problem belongs to one of C predetermined classes, i.e. $y \in \{1, \dots, C\}$

5.2 Classification Problem

Given observations (input) $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, $\mathbf{x}_i \in \mathbb{R}^D$ (each input is a D -dimensional feature vector), a set of possible classes $\mathcal{C} = \{1, \dots, C\}$ and labels (output) $\mathbf{y} = \{y_1, y_2, \dots, y_n\}$, $y_i \in \mathcal{C}$, the task of linear classification is to find a function $f: \mathbb{R}^D \rightarrow \mathcal{C}$ that maps the observation $\mathbf{x}_i$ to class label y_i

$$y_i = f(\mathbf{x}_i), \quad \text{for } i \in \{1, \dots, N\}$$

Differences between Regression and Classification There are some differences between them:

1. the output of classification belongs to a set of possible classes, while the output of regression is a real number.
2. we call the output of regression the targets and the output of classification labels.

5.3 Binary Classification

Try to find a hyperplane that separates the data from the two classes $\mathcal{C} = \{0, 1\}$ (notice that the class labels for binary classification are denoted as 0 and 1). The hyperplane can be defined by a normal vector $\mathbf{w}$ and an offset term w_0 , thus has the form

$$\mathbf{w}^T \mathbf{x} + w_0 = \begin{cases} 0, & \text{if } \mathbf{x} \text{ on the plane} \\ > 0, & \text{if } \mathbf{x} \text{ on normal's side} \\ < 0, & \text{else} \end{cases}$$

with $\frac{-w_0}{\|\mathbf{w}\|}$ the distance from hyperplane to the origin and $\frac{y(\mathbf{x})}{\|\mathbf{w}\|}$ the distance from the data point $\mathbf{x}$ to the hyperplane. The normal vector should be drawn perpendicularly on the hyperplane. The orientation of the hyperplane is thus controlled by the normal vector $\mathbf{w}$.

If we can separate a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$ through a hyperplane for which all $\mathbf{x}_i$ with $y_i = 0$ are on one side of the hyperplane and all $\mathbf{x}_i$ with $y_i = 1$ on the other side, then the dataset $\mathcal{D}$ is called **linearly separable**.

Perceptron One of the oldest methods for binary classification. Choose the function to be a step function

$$f(t) = \begin{cases} 1, & \text{if } t > 0 \\ 0, & \text{else} \end{cases}$$

So the output of the perceptron is $\tilde{y} = f(\mathbf{w}^T \mathbf{x} + w_0)$.

Learning rules for the perceptron:

Step 1: initialize the parameter $\mathbf{w}, w_0$ to arbitrary value, i.e. pick a hyperplane at arbitrary position, e.g. zero vector: $\mathbf{w}, w_0 \leftarrow \mathbf{0}$

Step 2: Use the misclassified sample $\mathbf{x}_i$ to update the hyperplane

$$\mathbf{w} \leftarrow \begin{cases} \mathbf{w} + \mathbf{x}_i, & \text{if } y_i = 1 \\ \mathbf{w} - \mathbf{x}_i, & \text{if } y_i = 0 \end{cases}$$
$$w_0 \leftarrow \begin{cases} w_0 + 1, & \text{if } y_i = 1 \\ w_0 - 1, & \text{if } y_i = 0 \end{cases}$$

we can also absorb the offset term to the normal vector by adding a constant 1 to it.

Step 3: iterate until all data are classified correctly

If the dataset $\mathcal{D}$ is linearly separable, after some iterations the perceptron will always find the hyperplane which separates the classes correctly.

5.4 Multiple Classes Classification

If we apply perceptron to multiple classes classification, we can use the one-versus-rest strategy. For each class, try a specific perceptron, each hyperplane $\mathcal{H}_i$ makes a binary decision whether it's class $\mathcal{C}_i$ or not class $\mathcal{C}_i$. For the intersection region, use majority vote to classify the data, but this might be the case that there exists no majority class after voting.

Multiclass Discriminant Decision regions separated by multiclass discriminant is always convex.

Multiclass discriminant tends to assign a data point to its closest class, i.e. assign the data point to the class where the distance to the decision boundary (the hyperplane) is the largest, since we are more confident about data points that are far away from the decision boundary. For each class, define a linear function of the form

$$f_c(\mathbf{x}) = \mathbf{w}_c^T \mathbf{x} + w_{0c}$$

The final decision rule is

$$\tilde{y} = \arg \max_{c \in \mathcal{C}} f_c(\mathbf{x})$$

that is, assign $\mathbf{x}$ to the class c which produces the highest $f_c(\mathbf{x})$.

Models like perceptron and multiclass discriminant are **hard-decision based** classifiers, they only work on linearly separable data set. If the data are not originally linear separable, we can use the basis function to map the data in another space where they can be linearly separated. We see that hard-decision based classifiers has several limitations:

- No measure of uncertainty
- Can't handle noisy data, i.e. very sensitive to noise
- Generalize badly
- Difficult to optimize

For these reasons, we want to model the distribution of the class label y given the data $\mathbf{x}$ by introducing probabilities.

5.5 Probabilistic Models for Classification

Probability that a data belongs to class c is

$$p(y = c|\mathbf{x}) = \frac{p(\mathbf{x}|y = c)p(y = c)}{p(\mathbf{x})}$$

If we know the posterior distribution $p(y|\mathbf{x})$, then we can assign a class to the data $\mathbf{x}$. A commonly used choice is to pick the class which has the largest posterior: $\tilde{y} = \arg \max_{c \in \mathcal{C}} p(y = c|\mathbf{x})$

There are two kinds of probabilistic model for classification problem. The Generative and the Discriminative model.

Generative Model the joint probability

Discriminative Directly model the posterior $p(y = c|\mathbf{x})$

5.6 Generative Model

Generative models obtain the posterior using the Bayes rule

$$p(\mathbf{x}, y = c) = \underbrace{p(\mathbf{x}|y = c)}_{\text{class-conditional}} \cdot \underbrace{p(y = c)}_{\text{class-prior}}$$

Applying a generative model includes two steps:

1. Choose a parametric model for the class conditional and class prior
2. Estimate the parameters of the model from the data using maximum likelihood

Then we can apply the learned model for generating new data. We can use categorical distribution for class prior

$$p(y = c) = \theta_c$$

and multivariate Gaussian for class conditionals

$$p(\mathbf{x}|y = c) = \mathcal{N}(\mathbf{x}|\mu_c, \Sigma) = \frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp\left\{-\frac{1}{2}(\mathbf{x} - \mu_c)^T \Sigma^{-1}(\mathbf{x} - \mu_c)\right\}$$

Notice: we use same covariance matrix Σ for each class! If each class has its own Σ_c , then the decision boundary will contain quadratic term. In total, we need to learn $\left(\frac{D}{2}\right)^2 + C \cdot D$ parameters (D is the dimension of sample vector and C the number of classes) since the covariance matrix is symmetric.

For two classes $\mathbf{C} = \{0, 1\}$, we have

$$\begin{aligned} p(y=1|\mathbf{x}) &= \frac{p(\mathbf{x}|y=1)p(y=1)}{p(\mathbf{x}|y=1) + p(\mathbf{x}|y=0)p(y=0)} \\ &= \frac{1}{1 + \exp(-a)} \\ &= \sigma(a) \rightarrow \text{the sigmoid function} \end{aligned}$$

where we define

$$a = \ln \frac{p(\mathbf{x}|y=1)p(y=1)}{p(\mathbf{x}|y=0)p(y=0)}$$

Linear Discriminant Analysis We'll see that the posterior can be seen as sigmoid applied on a linear function of $\mathbf{x}$ if all $\Sigma_c = \Sigma$, i.e. the same covariance for every class. To show that, plug in the Gaussian class-conditional and let class-prior temporarily aside

$$\begin{aligned} a &= \ln \frac{p(\mathbf{x}|y=1)p(y=1)}{p(\mathbf{x}|y=0)p(y=0)} \\ &= \ln \left[-\frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp \left(-\frac{1}{2}(\mathbf{x} - \mu_1)^T \Sigma^{-1}(\mathbf{x} - \mu_1) \right) \right] \\ &\quad - \ln \left[-\frac{1}{(2\pi)^{D/2}|\Sigma|^{1/2}} \exp \left(-\frac{1}{2}(\mathbf{x} - \mu_0)^T \Sigma^{-1}(\mathbf{x} - \mu_0) \right) \right] \\ &\quad + \ln p(y=1) - \ln p(y=0) \\ &= -\frac{1}{2}(\mathbf{x} - \mu_1)^T \Sigma^{-1}(\mathbf{x} - \mu_1) + \frac{1}{2}(\mathbf{x} - \mu_0)^T \Sigma^{-1}(\mathbf{x} - \mu_0) \\ &\quad + \ln p(y=1) - \ln p(y=0) \\ &= -\frac{1}{2}(\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mathbf{x}^T \Sigma^{-1} \mu_1 + \mu_1^T \Sigma^{-1} \mu_1) + \\ &\quad \frac{1}{2}(\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mathbf{x}^T \Sigma^{-1} \mu_0 + \mu_0^T \Sigma^{-1} \mu_0) + \ln p(y=1) \\ &\quad - \ln p(y=0) \\ &= \mathbf{x}^T \Sigma^{-1} \mu_1 - \mathbf{x}^T \Sigma^{-1} \mu_0 - \frac{1}{2} \mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_0^T \Sigma^{-1} \mu_0 \\ &\quad + \ln \frac{p(y=1)}{p(y=0)} \\ &= \mathbf{w}^T \mathbf{x} + w_0 \end{aligned}$$

where we define

$$\begin{aligned} \mathbf{w} &= \Sigma^{-1}(\mu_1 - \mu_0) \\ w_0 &= -\frac{1}{2} \mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_0^T \Sigma^{-1} \mu_0 + \ln \frac{p(y=1)}{p(y=0)} \end{aligned}$$

If each class has its own covariance Σ_c , we will get

$$\begin{aligned} a &= \frac{1}{2} \mathbf{x}^T [\Sigma_0^{-1} - \Sigma_1^{-1}] \mathbf{x} + \mathbf{x}^T [\Sigma_1^{-1} \mu_1 - \Sigma_0^{-1} \mu_0] \\ &\quad - \frac{1}{2} \mu_1^T \Sigma_1^{-1} \mu_1 + \frac{1}{2} \mu_0^T \Sigma_0^{-1} \mu_0 + \ln \frac{p(y=1)}{p(y=0)} + \frac{1}{2} \ln \frac{|\Sigma_0|}{|\Sigma_1|} \\ &= \mathbf{x}^T \mathbf{W}_2 \mathbf{x} + \mathbf{w}_1^T \mathbf{x} + w_0 \end{aligned}$$

where we define

$$\begin{aligned} \mathbf{W}_2 &= \frac{1}{2} [\Sigma_0^{-1} - \Sigma_1^{-1}] \\ \mathbf{w}_1 &= \Sigma_1^{-1} \mu_1 - \Sigma_0^{-1} \mu_0 \\ w_0 &= -\frac{1}{2} \mu_1^T \Sigma_1^{-1} \mu_1 + \frac{1}{2} \mu_0^T \Sigma_0^{-1} \mu_0 + \ln \frac{p(y=1)}{p(y=0)} + \frac{1}{2} \ln \frac{|\Sigma_0|}{|\Sigma_1|} \end{aligned}$$

We now see that if we assume same covariance for each class, then we get a linear decision boundary and if the covariance is not identical, we get a quadratic decision boundary.

LDA for $C = 2$ classes: for the identical covariance case, we see that the posterior distribution is a sigmoid of a linear function of $\mathbf{x}$

$$\begin{aligned} p(y = 1|\mathbf{x}) &= \frac{1}{1 + \exp(-(\mathbf{w}^T \mathbf{x} + w_0))} \\ &= \sigma(\mathbf{w}^T \mathbf{x} + w_0) \end{aligned}$$

LDA for $C > 2$ classes: the posterior is

$$\begin{aligned} p(y = c|\mathbf{x}) &= \frac{p(\mathbf{x}|y = c)p(y = c)}{\sum_{c'=1}^C p(\mathbf{x}|y = c')p(y = c')} \\ &= \frac{\exp(\mathbf{w}_c^T \mathbf{x} + w_{c0})}{\sum_{c'=1}^C \exp(\mathbf{w}_{c'}^T \mathbf{x} + w_{c'0})} \\ &= \text{softmax}(\mathbf{w}_c^T \mathbf{x} + w_{c0}) \end{aligned}$$

where we define

$$\begin{aligned} \mathbf{w}_c &= \Sigma^{-1} \mu_c \\ w_{c0} &= -\frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c + \ln p(y = c) \end{aligned}$$

5.7 Discriminative Model

From LDA we see that parameter of generative models $\mathbf{w}, w_0$ are dependent on the parameters of Gaussian class-conditionals μ_0, μ_1, Σ . For discriminative model, we can choose $\mathbf{w}, w_0$ freely.

Logistic Regression Always remember that logistic regression is used for binary classification!

The difference is that logistic regression assumes that $y \sim \text{Bernoulli}(\sigma(\mathbf{w}^T \mathbf{x} + w_0))$, while linear regression assumes that $y \sim \mathcal{N}(f_{\mathbf{w}}(\mathbf{x}), \beta^{-1})$.

The task of logistic regression is somehow similar to linear regression. We want to find a set of $\mathbf{w}$ that fits the data the best.

Same as the probabilistic perspective of linear regression, for logistic regression we can also write down the likelihood function and use maximum likelihood to find the best $\mathbf{w}$

$$\begin{aligned} p(\mathbf{y}|\mathbf{w}, \mathbf{X}) &= \prod_{i=1}^N p(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \prod_{i=1}^N p(y = 1|\mathbf{x}_i, \mathbf{w})^{y_i} (1 - p(y = 1|\mathbf{x}_i, \mathbf{w}))^{1-y_i} \\ &= \prod_{i=1}^N \sigma(\mathbf{w}^T \mathbf{x}_i)^{y_i} (1 - \sigma(\mathbf{w}^T \mathbf{x}_i))^{1-y_i} \end{aligned}$$

Again, we define the negative log-likelihood as the error function $E(\mathbf{w})$, and we can obtain the optimal $\mathbf{w}^*$ by minimizing the error function $\mathbf{w}^* = \arg \min E(\mathbf{w})$

$$\begin{aligned} E(\mathbf{w}) &= -\ln p(\mathbf{y}|\mathbf{w}, \mathbf{X}) \\ &= -\sum_{i=1}^N (y_i \ln \sigma(\mathbf{w}^T \mathbf{x}_i) + (1 - y_i) \ln(1 - \sigma(\mathbf{w}^T \mathbf{x}_i))) \\ &\Rightarrow \text{binary cross entropy} \end{aligned}$$

Unfortunately, unlike linear regression, there exist **NO** close-form solution of optimal weights vector for logistic regression, because the error function of logistic regression is not convex. To find $\mathbf{w}^*$, we need to apply numeric methods like gradient descend etc.

Because we use maximum likelihood method to find the $\mathbf{w}^*$, this leads easily to overfitting issue. To prevent overfitting, we can always add regularizations to the error function and penalize large weights

$$E(\mathbf{w}) = -\ln p(\mathbf{y}|\mathbf{w}, \mathbf{X}) + \lambda \|\mathbf{w}\|^2$$

Logistic Regression for $C = 2$ classes:

$$\begin{aligned} p(\mathbf{y}|\mathbf{w}, \mathbf{X}) &= \prod_{i=1}^N p(y_i|\mathbf{x}_i, \mathbf{w}) \\ &= \prod_{i=1}^N p(y_i = 1|\mathbf{x}_i, \mathbf{w})^{y_i} (1 - p(y_i = 1|\mathbf{x}_i, \mathbf{w}))^{1-y_i} \\ &= \prod_{i=1}^N \sigma(\mathbf{w}^T \mathbf{x}_i)^{y_i} (1 - \sigma(\mathbf{w}^T \mathbf{x}_i))^{1-y_i} \end{aligned}$$

Logistic Regression for $C > 2$ classes:

$$p(y = c|\mathbf{x}) = \frac{\exp(\mathbf{w}_c^T \mathbf{x})}{\sum_{c'} \exp(\mathbf{w}_{c'}^T \mathbf{x})} = \text{softmax}(\mathbf{w}_c^T \mathbf{x})$$

with error function

$$\begin{aligned} E(\mathbf{w}) &= -\ln p(\mathbf{Y}|\mathbf{w}, \mathbf{X}) \\ &= -\sum_{i=1}^N \sum_{c=1}^C y_{ic} \ln p(y_i = c|\mathbf{x}_i, \mathbf{w}) \\ &= -\sum_{i=1}^N \sum_{c=1}^C y_{ic} \ln \frac{\exp(\mathbf{w}_c^T \mathbf{x}_i)}{\sum_{c'} \exp(\mathbf{w}_{c'}^T \mathbf{x}_i)} \\ &\Rightarrow \text{cross entropy} \end{aligned}$$

with $\mathbf{Y} \in \{0, 1\}^{N \times D}$ in a one-hot-encoding fashion, i.e. each row of $\mathbf{Y}$ is a label one-hot-vector

$$y_{ic} = \begin{cases} 1, & \text{if sample } i \text{ belongs to class } c \\ 0, & \text{else} \end{cases}$$

5.8 Generative vs Discriminative Model

- In general, discriminative models work better on pure classification tasks than generative models
- Generative models assume Gaussian class-conditionals, and if this assumption holds, generative models work reasonably better than discriminative models. On the other hand if the assumption is violated, generative models become fragile
- Generative models still don't perform well for high-dimensional/strongly correlated data
- Generative models are good at handling missing data, detecting outliers and generating new data. Due to the i.i.d assumption we can always kick out those terms of missing/noisy data from the likelihood function, just simply ignore their contribution to the likelihood and we're good

6 Optimization

Recall from logistic regression, we were able to write down the error function of logistic regression, but unfortunately there exists no close-form solution of optimal weight vector $\mathbf{w}^*$, i.e. we can't obtain $\mathbf{w}^*$ by simply taking derivative of error function and setting it to zero. We need numeric methods to do that.

The task of optimization is to find a solution θ^* which minimizes the objective function f :

$$\theta^* = \arg \min_{\theta} f(\theta)$$

6.1 Convex Set and Convex Function

Convex Set Intuitively, randomly pick two points in a set and connect them with a straight line, if all points on the line are also in the set, then the set is called convex. Mathematically, X is a convex set iff for all $x, y \in X$, we have

$$\lambda x + (1 - \lambda)y \in X, \quad \forall \lambda \in [0, 1]$$

Convex Function $f(x)$ is a convex function on a convex set X iff for all $x, y \in X$, we have

$$\lambda f(x) + (1 - \lambda)f(y) \geq f(\lambda x + (1 - \lambda)y), \quad \forall \lambda \in [0, 1]$$

Properties of convex function

- Region above a convex function is convex
- Convex functions don't have local minima
- Each local minimum is a global minimum

First order convexity conditions For $f : \mathcal{X} \rightarrow \mathbb{R}$ a first order differentiable function that maps a convex set $\mathcal{X}$ to real number, then f is convex iff for $x, y \in \mathbb{R}$ if we have

$$f(\mathbf{y}) \geq f(\mathbf{x}) + (\mathbf{y} - \mathbf{x})^T \nabla f(\mathbf{x})$$

We gonna need this theorem for proving the third property of convex function

Vertices and Convex Hull A vertex $\mathbf{x} \in \mathcal{X}$ is called a vertex of the convex set $\mathcal{X}$ if it can't be extrapolated within the convex set, i.e. $\mathbf{x}$ is somehow the corner point of the convex set

$$(\lambda \mathbf{x} + (1 - \lambda)\mathbf{x}') \notin \mathcal{X} \text{ for } \lambda > 1 \text{ for } \forall \mathbf{x}' \in \mathcal{X}, \mathbf{x}' \neq \mathbf{x}$$

A convex hull $\text{Conv}(\mathcal{X})$ is expressed by the linear combination (convex combination) of points within the convex set $\mathcal{X}$

$$\begin{aligned} \text{Conv}(\mathbf{X}) \\ = \left\{ \sum_{i=1}^n \alpha_i \cdot \mathbf{x}_i \mid \mathbf{x}_i \in \mathcal{X}, n \in \mathbb{N}, \sum \alpha_i = 1, \alpha_i \geq 0 \right\} \end{aligned}$$

remember convex hull is a concept to convex set. Intuitively, given a two dimensional convex set, the convex hull is then a convex polygon that joins all outermost points and includes all points in the convex set inside itself.

We have following properties

- Convex hull of a set is convex set, no matter whether the original set is convex or not, i.e. even though the set is originally concave, its convex hull is always convex.
- Vertices of convex hull is always a subset of the convex set itself $\text{Ve}(\text{Conv}(\mathcal{X})) \subseteq \mathcal{X}$.
- Maximum over a convex function on a convex set is obtained on a vertex. Intuitively we can just imagine that the convex set is a deep hole and the vertex is the closest to the ground surface. So to find the maximum of the finite convex set, we only need to test the vertices of the convex set and compare their values and, a step further, we only need to examine the vertices of its convex hull, so we don't need to check the inner points which are definitely not maximum.

Verifying convexity for functions

- First order convexity conditions
- A twice differentiable function (has one or multiple variables) is convex on an interval if and only if the second-derivative is non-negative (one variable) or the Hessian matrix is positive semi-definite
- Operations that preserve convexity

Convexity preserving operations for functions Assume $f_1 : \mathbb{R}^d \rightarrow \mathbb{R}$ and $f_2 : \mathbb{R}^d \rightarrow \mathbb{R}$ are convex functions and $g : \mathbb{R}^d \rightarrow \mathbb{R}$ is concave function, then we have following convexity preserving operations

- $h(\mathbf{x}) = f_1(\mathbf{x}) + f_2(\mathbf{x})$ is convex
- $h(\mathbf{x}) = \max\{f_1(\mathbf{x}), f_2(\mathbf{x})\}$ is convex
- $h(\mathbf{x}) = c \cdot f_1(\mathbf{x})$ is convex if $c \geq 0$
- $h(\mathbf{x}) = c \cdot g(\mathbf{x})$ is convex if $c \leq 0$

- $h(\mathbf{x}) = f_1(\mathbf{A}\mathbf{x} + \mathbf{b})$ is convex $\mathbf{A}$ is a matrix and $\mathbf{b}$ is a vector
- $h(\mathbf{x}) = m(f_1(\mathbf{x}))$ is convex if $m: \mathbb{R} \rightarrow \mathbb{R}$ is convex and nondecreasing
- $h(x) = \text{const}$ and $h(\mathbf{x}) = \mathbf{x}^T \cdot \mathbf{b}$ are convex operations
- $h(x) = e^x$ is convex operation

We shall notice that we may need to find the exact interval where the functions are convex on

Verifying convexity for sets

- Check whether the definition of convex set holds

$$(\lambda \mathbf{x} + (1 - \lambda) \mathbf{x}') \notin \mathcal{X} \text{ for } \lambda > 1 \text{ for } \forall \mathbf{x}' \in \mathcal{X}, \mathbf{x}' \neq \mathbf{x}$$

- Examine the intersection rule, i.e. if $\mathcal{A}$ and $\mathcal{B}$ are both convex sets, then $\mathcal{A} \cap \mathcal{B}$ is also a convex set.

6.2 Convex Optimization

For a specific objective function $f(\theta)$, we examine its convexity first. If f is indeed a convex function, then we can easily compute the derivative and set it to zero, i.e. $f'(\theta) = 0$ to obtain the θ that minimizes the objective function and, if there're no constraints, we're done, just like what we did in linear regression with least-square error function. If there're constraints on θ , we need to use the techniques of solving constrained optimization problems.

Actually the objective function may not be differentiable on whole domain (e.g. $f(x) = |x|$ is not differentiable at $x = 0$), in this case we may want to use subgradients or turn to discrete optimization problem. And, in very bad situation, f is not even convex, if that happens, we may want to do convex relaxations, to see whether f is convex in some variables.

The problem is, as mentioned before in logistic regression, the objective function is convex doesn't ensure that there exists a close-form solution of the optimal θ , e.g. we don't have close-form solution for logistic regression, so we may want to try the numerical approaches

One-dimensional problems Use numerical approach to solve one-dimensional problems: binary search

Algorithm 1 Binary Search - Output $x = \frac{A+B}{2}$

Require: a, b , Precision ϵ

```

while  $(B - A) \min(|f'(A)|, |f'(B)|) > \epsilon$  do
  if  $f'(\frac{A+B}{2}) > 0$  then
     $B = \frac{A+B}{2}$ 
  else
     $A = \frac{A+B}{2}$ 
  end if
end while
```

6.3 Gradient Descent

Key ideas: gradient points into steepest ascent direction and locally, the gradient is a good approximation of the objective function.

Gradient Descent and similar techniques are called **first-order** optimization techniques since they only use gradient information (i.e. first order derivative) to update the parameters.

Algorithm 2 Gradient Descent

Require: a starting point $\theta \in \text{dom}(f)$

```

while stop criterion is not fulfilled do
   $\Delta\theta = -\nabla f(\theta)$ 
  Line search  $t = \arg \min_{t \geq 0} f(\theta + t \cdot \Delta\theta)$ 
  Update  $\theta = \theta + t\Delta\theta$ 
end while
```

Gradient Descent converges linearly, i.e. error goes down exponentially with number of iteration.

Line search types Line search determines how much do we step into the gradient direction.

Exact line search:

$$t = \arg \min_{t \geq 0} f(\theta + t \cdot \Delta \theta)$$

Backtracking line search: with parameter $\alpha \in (0, \frac{1}{2})$ and $\beta \in (0, 1)$. Start at $t = 1$, repeat $t = \beta t$ (t gets smaller every iteration) until

$$f(\theta + t \cdot \Delta \theta) < f(\theta) + t \cdot \alpha \cdot \nabla f(\theta)^T \Delta \theta$$

graphically it's just a $\alpha\%$ for the gradient slope

Backtracking line search has this zick-zack shape of parameter update and therefore needs more iteration to converge, while exact line search needs more computation time at each iteration but the overall number of iteration is smaller.

Distributed/Parallel implementation To speed up the gradient computation, we can decompose the gradient based on the sum rule.

- Distribute data over several Machines
- Compute partial gradient on each machine Parallel
- Aggregate the partial gradients to the final one
- Communicate the final gradient back to all machines to do the parameter update

Distributed implementation leads to multiple passes through dataset for each iteration. But line search is very expensive, so we may want to avoid line search and do single pass through dataset for each iteration

$$\theta_{t+1} \leftarrow \theta_t - \tau \cdot \nabla f(\theta)$$

with τ called learning rate. If τ is picked too small, we might have a very slow convergence and stuck at local minima or saddle point, and if τ is too big, the algorithm might oscillate and not converge.

Learning rate adaption Let the learning rate to decrease at each iteration, e.g. $\tau_{t+1} \leftarrow \alpha \cdot \tau_t$ for $0 < \alpha < 1$. Big learning rate at start and small learning rate after many iterations, so the algorithm converges when $\lim_{t \rightarrow \infty} \tau_t = 0$.

Stochastic Gradient Descent For large scale data or high-dimensional problems we usually use first-order optimization techniques like gradient descent to solve the problem. The truth is real-world data are so computationally expensive even first-order techniques can't handle the issue.

The idea is we can use a minibatch of entire data to compute a noisy gradient and use it for parameter updating.

Steps of SGD

- Randomly pick a small subset $\mathcal{S}$ from the entire data $\mathcal{D}$, i.e. the so called mini-batch
- Compute the gradient based on mini-batch
- Do the update $\theta_{t+1} \leftarrow \theta_t - \tau \cdot \frac{n}{|\mathcal{S}|} \cdot \nabla f(\theta_t)$
- Pick a new mini-batch and repeat

Larger mini-batches lead to more stable gradients (i.e. smaller variance in the estimated gradient). Each iteration of SGD is shorter than standard gradient descent, but in total it will need more iterations to converge

6.4 Other Optimization Approaches

Momentum and AdaGrad both consider historical gradients while updating the parameter.

Momentum:

1. Integrate history of previous gradients into parameter update
2. As long as gradients point to the same direction, the search accelerates

$$m_t \leftarrow \tau \cdot \nabla f(\theta_t) + \gamma \cdot m_{t-1}$$

$$\theta_{t+1} \leftarrow \theta_t - m_t$$

AdaGrad:

1. Different learning rate per parameter
2. Learning rate depends on accumulated strength of all previously computed gradients
3. large parameter updates lead to small learning rates

Adam:

1. estimate the first moment of the gradient (the mean)

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla f(\theta_t)$$

2. estimate the second moment of the gradient (the variance)

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla f(\theta_t))^2$$

3. Modify first and second moment to avoid zero bias

$$\tilde{m}_t = \frac{m_t}{1 - \beta_1^T}$$

$$\tilde{v}_t = \frac{v_t}{1 - \beta_2^T}$$

4. Update the parameter with modified moments

$$\theta_{t+1} = \theta_t - \frac{\tau}{\sqrt{\tilde{v}_t} + \epsilon}$$

6.5 Newton Method

Recall that gradient descent and similar approaches are first-order optimization techniques that only use first derivative to update the parameter. Newton method on the contrary represents the **higher-order** optimization techniques, which uses both first-order and second-order derivatives to update the parameters.

Prerequisite of Newton method

- Objective function f is convex
- Second-order derivatives are non-negative, i.e. semi-positive definite Hessian matrix H

Steps of Newton Method

- Approximate f with second-order Taylor expansion of f at point θ_t

$$f(\theta_t + \delta) = f(\theta_t) + \delta^T \cdot \nabla f(\theta_t) + \frac{1}{2} \delta^T \nabla^2 f(\theta_t) \delta + O(\delta^3)$$

- Instead of minimizing f , we minimize the approximation

$$\theta_{t+1} \leftarrow \theta_t - \underbrace{[\nabla^2 f(\theta_t)]^{-1}}_{\text{inverse of } H} \nabla f(\theta_t)$$

By the way, gradient descent can be seen as minimizing first-order Taylor approximation.

Properties of Newton Method

- Newton method rescales the space
- Newton method converges in less steps than gradient descent, but also need more information (i.e. the need of computing Hessian matrix)
- Like gradient descent, a distributed computation for Hessian is possible
- Use Newton method only for low dimensional problems, since higher-order optimization techniques are expensive for high dimensional problems. So for large scale data or high dimensional problems, we use first-order techniques, e.g. gradient descent and similar approaches

7 Constrained Optimization

7.1 Inequality Constraints

Given primal problem $f_0 : \mathbb{R}^D \rightarrow \mathbb{R}$ and constraints $f_i : \mathbb{R}^D \rightarrow \mathbb{R}$, solve

$$\begin{aligned} & \text{minimize} && f_0(\theta) \\ & \text{subject to} && f_i(\theta) \leq 0, \quad i = 1, \dots, M \end{aligned}$$

and a point θ is called feasible if and only if it satisfies all constraints $f_i(\theta) \leq 0, i = 1, \dots, M$ of the optimization problem. Let p^* be the minimum and the θ^* the minimizer, so we have $p^* = f_0(\theta^*)$

Notice that there exist also equality constraints.

Linear Programming (LP) The primal function is a linear function and the constraints are all linear constraints.

$$\begin{aligned} & \text{minimize} && \mathbf{c}^T \theta \\ & \text{subject to} && \mathbf{A}\theta - \mathbf{b} \leq 0, \theta_i \geq 0 \quad \forall i \in [1, D] \end{aligned}$$

Quadratic Programming (QP) The primal function is a quadratic function and the constraints are all linear constraints.

$$\begin{aligned} & \text{minimize} && \frac{1}{2} \theta^T \mathbf{Q} \theta + \mathbf{c}^T \theta \\ & \text{subject to} && \mathbf{A}\theta - \mathbf{b} \leq 0 \end{aligned}$$

if $\mathbf{Q}$ is positive semi-definite, then the primal function is convex.

7.2 Lagrangian

Constraints help us to reduce the search space of the optimization problem. But if the constraints themselves are complex, the problem is not getting any easier. Thus we want to transfer the constrained optimization problem into non-constrained optimization problem.

Lagrangian L is defined as: $\mathbb{R}^D \times \mathbb{R}^M \rightarrow \mathbb{R}$, which linearly combines the primal function and constraints together

$$L(\theta, \alpha) = f_0(\theta) + \underbrace{\sum_{i=1}^M \alpha_i f_i(\theta)}_{\leq 0}$$

with $\alpha_i \geq 0$ as the **Lagrange multiplier**. We see that in Lagrangian, the constraint term is always smaller equals to zero. Means that if the constraints term is zero (i.e. non-constrained problem), the Lagrangian has the same optimum as the primal function $f_0(\theta)$. If the constraints term is smaller than zero and the primal function is convex, we see that the optimum of the primal function is not going to be smaller than the optimum of the Lagrangian, so Lagrangian is a **lower bound** on the optimal value of the constrained problem.

Take the gradient of the Lagrangian L w.r.t θ gives us the optimality criterion of θ .

$$\nabla_{\theta^*} L(\theta^*, \alpha) = \nabla f_0(\theta^*) + \sum_{i=1}^M \alpha_i \nabla f_i(\theta^*) = 0$$

Lagrangian dual function The Lagrange dual function $g : \mathbb{R}^M \rightarrow \mathbb{R}$ is the minimum of the Lagrangian over θ given θ

$$g(\alpha) = \min_{\theta \in \mathbb{R}^D} L(\theta, \alpha) = \min_{\theta \in \mathbb{R}^D} \left(f_0(\theta) + \sum_{i=1}^M \alpha_i f_i(\theta) \right)$$

Given the Lagrangian dual function, we want to find out which α maximizes the $g(\alpha)$, we want to find an optimal solution for the constrained optimization problem which is the closest to the optimum of the primal problem. The maximum d^* of the Lagrangian dual problem is the best lower bound on p^* that we can achieve by using the Lagrangian

$$\begin{aligned} & \text{maximize} && g(\alpha) = \min_{\theta} L(\theta, \alpha) \\ & \text{subject to} && \alpha_i \geq 0, i = 1, \dots, M \end{aligned}$$

7.3 Duality and Recipe for solving COP

Weak duality Since the optimum of dual problem d^* is a lower bound of the optimum of primal problem p^* , we always have weak duality hold

$$d^* \leq p^*$$

The difference between d^* and p^* is called duality gap. Under certain conditions we can make the duality gap to be zero.

Strong duality Always remember that strong duality only holds when certain conditions are fulfilled

$$d^* = p^*$$

i.e. the solution to the Lagrangian dual problem is a solution of the primal problem. If strong duality holds, we have

$$L(\theta^*, \alpha^*) = g(\alpha^*) = \min_{\theta} L(\theta, \alpha^*)$$

So if the primal problem is convex, to find the primal optimizer θ^* , what we need to do is to find the optimizer of the dual problem α^* and plug it in the Lagrangian, then optimize Lagrangian over θ

$$\theta^* = \arg \min_{\theta} L(\theta, \alpha^*)$$

Recipe Follow the recipe to solve the constrained optimization problem.

Given the constrained optimization problem

$$\begin{aligned} & \text{minimize} && f_0(\theta) \\ & \text{subject to} && f_i(\theta) \leq 0 \quad i = 1, \dots, M \end{aligned}$$

where the primal function f_0 and constraints $f_1, \dots, f_M$ are all convex. The recipe has following steps

Step 1: Write down the Lagrangian

$$L(\theta, \alpha) = f_0(\theta) + \sum_{i=1}^M \alpha_i f_i(\theta)$$

Step 2: Take derivative of Lagrangian w.r.t θ and set it equal to zero to obtain the optimizer of Lagrangian θ^*

$$\theta^*(\alpha) = \arg \min_{\theta} L(\theta, \alpha) \Leftrightarrow \nabla_{\theta} L(\theta, \alpha) = 0$$

Step 3: Plug the optimizer $\theta^*(\alpha)$ into Lagrangian to get dual function $g(\alpha)$, and maximize the dual function w.r.t α

$$\begin{aligned} & \text{maximize} && g(\alpha) = L(\theta^*(\alpha), \alpha) \\ & \text{subject to} && \alpha_i \geq 0, i = 1, \dots, M \end{aligned}$$

We should notice that if we maximize the primal function, we need to flip the sign while building the Lagrangian function, i.e.

$$\text{maximize } f_0(\theta) = \text{minimize } -f_0(\theta)$$

Slater's constraint qualification This tells when the strong duality holds (i.e. the duality gap is zero).

1. constraints $f_0, f_1, \dots, f_M$ must be convexity
2. there exists a $\theta \in \mathbb{R}^D$ such that each constraint fulfills, i.e. $f_i(\theta) < 0$, strictly smaller than zero, or (b) the constraint is affine, i.e. $f_i(\theta) = \mathbf{w}_i^T \theta + b_i \leq 0$

7.4 KKT condition

Karush-Kuhn-Tucker (KKT) condition tells when θ^* and α^* are indeed optimizers to their corresponding optimization problems

$$\begin{array}{ll} f_i(\theta^*) \leq 0 & \text{primal feasibility} \\ \alpha_i^* \geq 0 & \text{dual feasibility} \\ \alpha_i^* f_i(\theta^*) = 0 & \text{complementary slackness} \\ \nabla_{\theta} L(\theta^*, \alpha^*) = 0 & \theta^* \text{ minimizes Lagrangian} \end{array}$$

for all $i = 1, \dots, M$.

7.5 Projected Gradient Descent

The problem of applying Gradient Descent to solve the constrained optimization problem is that the solution might not be in the valid region $\mathcal{X}$, i.e.

$$\theta^t \in \mathcal{X}, \text{ but } \theta^{t+1} \leftarrow \theta^t - \tau \nabla f(\theta^t) \notin \mathcal{X} \quad \text{for all } \tau > 0$$

The idea of Projected Gradient Descent is to project the new point back on the convex set $\mathcal{X}$

$$\theta^{t+1} \leftarrow \pi_{\mathcal{X}}(\theta^t - \tau \nabla f(\theta^t))$$

where $\pi_{\mathcal{X}}(\mathbf{p}) = \arg \min_{\theta \in \mathcal{X}} \|\theta - \mathbf{p}\|^2$, that is, find the closest valid point on the convex set $\mathcal{X}$. The projection itself is convex, and if all the constraints are linear, then we have a quadratic programming task which we can use standard solvers.

We also can do the projection in a more efficient way by projecting back on a subset of valid region $\mathcal{X}$.

- Projection onto box-constraints $\mathcal{X} = \{\theta \in \mathbb{R}^d \mid \forall i \in [1, d] : l_i \leq \theta_i \leq u_i\}$

$$\pi_{\mathcal{X}}(\mathbf{p}) = \min(\max(l_i, p_i), u_i)$$

- Projection onto L2-ball $X = \{\theta \in \mathbb{R}^d \mid \|\theta\|_2 \leq c\}$

$$\pi_{\mathcal{X}}(\mathbf{p}) = \begin{cases} \mathbf{p} & \text{if } \|\mathbf{p}\| \leq c \\ \frac{c}{\|\mathbf{p}\|_2} \mathbf{p} & \text{otherwise} \end{cases}$$

- Projection onto L1-ball $\rightarrow$ Linear time algorithm
- Projection onto L1-ball and box-constraints

Discussion of projected gradient descent

- Often used for solving large-scale constrained optimization problems
- Highly efficient if projection can be evaluated efficiently
- Each step leads to a feasible solution
- Like standard gradient descent, choice of learning rate etc. required
- Projected gradient descent is a special case of so called **proximal methods**

8 SVM

Support vector machine can be used as binary classifier which separates both classes with maximum margin. SVM is not a probabilistic model!

8.1 Hyperplane

Recall that from linear classification, a hyperplane can be defined with a normal vector, a offset term and a point on the hyperplane, i.e. $\mathbf{w}^T \mathbf{x} + b = 0$. For all points which lie exactly on the hyperplane, we have $\mathbf{w}^T \mathbf{x} + b = 0$, for points which lie on each side of the hyperplane, we have $\mathbf{w}^T \mathbf{x} + b < 0$ and $\mathbf{w}^T \mathbf{x} + b > 0$. So the class of a point $\mathbf{x}$ is given by

$$h(\mathbf{x}) = \text{sgn}(\mathbf{w}^T \mathbf{x} + b)$$

where

$$\text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

We add two more hyperplanes that are parallel to the original hyperplane and require that no training points must lie between those hyperplanes. For all points from class +1 we have $\mathbf{w}^T \mathbf{x} + (b - s) > 0$ and all points from class -1 we have $\mathbf{w}^T \mathbf{x} + (b + s) < 0$. Notice that we denote these two classes as ± 1 for a particular reason,

we shall see the reason very soon. Minus s corresponds to moving the original hyperplane upward because the distance s is signed distance. We have the margin

$$m = d_{+1} - d_{-1} = \frac{2s}{\|\mathbf{w}\|}$$

depends on the ratio of s and $\|\mathbf{w}\|$. So for convenience we set $s = 1$ and get $m = 2/\|\mathbf{w}\|$. Notice that although the distance from the origin to the middle hyperplane also only depends on the ratio of offset term and normal vector, as in $d = -b/\|\mathbf{w}\|$ we **can't** set the offset term to 1 for convenience, otherwise it would link the distance d to the size of the margin m .

As announced earlier, we set the label of the two classes as $y_i \in \{-1, +1\}$ assigned to $\mathbf{x}_i$, after set the $s = 1$ we get

$$\begin{aligned}\mathbf{w}^T \mathbf{x}_i + b &\geq 1 & \text{for } y_i = +1 \\ \mathbf{w}^T \mathbf{x}_i + b &\leq -1 & \text{for } y_i = -1\end{aligned}$$

Because we denote the class labels as ± 1 , we can unify these two constraints as

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \text{for all } i$$

If these constraints are fulfilled the margin is

$$m = \frac{2}{\|\mathbf{w}\|} = \frac{2}{\sqrt{\mathbf{w}^T \mathbf{w}}}$$

so maximize the margin is the same as minimize $\mathbf{w}^T \mathbf{w}$. So we have the primal function

$$f_0(\mathbf{w}^T, b) = \frac{1}{2} \mathbf{w}^T \mathbf{w}$$

we add the constant coefficient $\frac{1}{2}$ just for derivative convenience.

8.2 Optimization Problem

The difference between SVM and logistic regression is (although they both deal with binary classification problems) that SVM tries to maximize the margin, so this is truly a constrained optimization problem where the primal problem is to maximize the margin and the constraints are $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ for all i . To find the separating hyperplane with the maximum margin we need to find $\{\mathbf{w}, b\}$ that

$$\begin{aligned}\text{minimize} \quad & f_0(\mathbf{w}, b) = \frac{1}{2} \mathbf{w}^T \mathbf{w} \\ \text{subject to} \quad & f_i(\mathbf{w}, b) = y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0 \\ & \text{for } i = 1, \dots, N\end{aligned}$$

with N denotes the number of points that needs to be classified. We can see that SVM is a constrained convex optimization problem (to be more specific, a quadratic programming problem with quadratic primal and linear constraints). We should also notice that in the definition of inequality constraints, we use $\leq$, above in SVM we have $\geq$, so we should put minus sign while writing the Lagrangian.

Since this is a constrained convex optimization problem, we can use the recipe to solve it.

Step 1: write down Lagrangian

$$L(\mathbf{w}, b, \alpha) = \underbrace{\frac{1}{2} \mathbf{w}^T \mathbf{w}}_{\text{primal}} - \underbrace{\sum_{i=1}^N \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1]}_{\text{constraints}}$$

Step 2: Minimize $L(\mathbf{w}, b, \alpha)$ w.r.t $\mathbf{w}$ and b

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b, \alpha) = \mathbf{w} - \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i = 0$$

so we have the Lagrangian optimizer

$$\mathbf{w}^* = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$$

minimizing the Lagrangian w.r.t b we have

$$\nabla_b L(\mathbf{w}, b, \alpha) = \sum_{i=1}^N \alpha_i y_i = 0$$

we see that there's no close-form solution for the Lagrangian optimizer b^* , but if we gonna plug b^* in the Lagrangian function to get the dual function, $\sum_{i=1}^N \alpha_i y_i = 0$ must hold, so we add this to the constraints.

Substitute the Lagrangian optimizers $\mathbf{w}^* = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$ back into $L(\mathbf{w}, b, \alpha)$

$$\begin{aligned} L(\mathbf{w}, b, \alpha) &= \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] \\ &= \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i y_i (\mathbf{w}^T \mathbf{x}_i + b) + \sum_{i=1}^N \alpha_i \\ &= \frac{1}{2} \sum_{i=1}^N (\alpha_i y_i \mathbf{x}_i)^T \sum_{j=1}^N (\alpha_j y_j \mathbf{x}_j) - \sum_{i=1}^N \alpha_i y_i \sum_{j=1}^N (\alpha_j y_j \mathbf{x}_j)^T \mathbf{x}_i \\ &\quad - \sum_{i=1}^N \alpha_i y_i b + \sum_{i=1}^N \alpha_i \\ &= \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j - \\ &\quad \underbrace{\sum_{i=1}^N \alpha_i y_i b}_{=0} + \sum_{i=1}^N \alpha_i \\ &= -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j + \sum_{i=1}^N \alpha_i \end{aligned}$$

where $\sum_{i=1}^N \alpha_i y_i = 0$ (come together when we compute the derivative w.r.t the bias b). Maximize it we then get the dual function

$$\begin{aligned} \text{maximize } g(\alpha) &= \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j \\ \text{subject to } &\sum_{i=1}^N \alpha_i y_i = 0 \\ &\alpha_i \geq 0, \forall i \in [1, N] \end{aligned}$$

We can also rewrite the dual function in matrix form

$$g(\alpha) = \frac{1}{2} \alpha^T \mathbf{Q} \alpha + \alpha^T \mathbf{1}_N$$

where $\mathbf{Q}$ is a symmetric negative semi-definite matrix (so that the dual function is convex), and constrains on α are linear. Since we maximize the dual function, we see that SVM is an example of quadratic programming. Algorithms like Sequential minimal optimization (SMO) are efficient for solving QP problems.

Solve the dual problem with QP solver we get the dual optimizer α_i^* (the optimizer of dual problem is a set of α , not just one). Plug in the Lagrangian optimizer we get $\mathbf{w}^*$

$$\mathbf{w}^* = \sum_{i=1}^N \alpha_i^* y_i \mathbf{x}_i$$

and the bias b^* , which we can easily recover from complementary slackness condition $\alpha_i^* f_i(\mathbf{w}, b) = 0$

$$\begin{aligned} f_i(\mathbf{w}, b) &= y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 = 0 \\ \Rightarrow b &= \frac{1}{y_i} - \mathbf{w}^T \mathbf{x}_i = y_i - \mathbf{w}^T \mathbf{x}_i \end{aligned}$$

for any vector $\mathbf{x}_i$ fulfills the constraint $\alpha_i \neq 0$.

Support Vector From complementary slackness we know

$$\alpha_i[y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0 \quad \text{for all } i$$

Training samples $\mathbf{x}_i$ with $\alpha_i \neq 0$ is called support vectors. They all lie on the margin, thus $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$. Only the model is trained, a significant proportion of the data points can be disregarded, i.e. we only need to retain those support vectors for constructing the hyperplane.

Classifying Recall that the class of a data point $\mathbf{x}$ is given by

$$h(\mathbf{x}) = \text{sgn}(\mathbf{w}^T \mathbf{x} + b)$$

substitute $\mathbf{w}$ with the Lagrangian optimizer $\mathbf{w}^*$ we get

$$h(\mathbf{x}) = \text{sgn}\left(\sum_{i=1}^N \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b\right)$$

So to classify a new data point after the model is trained, we only need to remember the fewing training samples $\mathbf{x}_i$ with $\alpha_i \neq 0$.

8.3 Soft Margin SVM

The discussions before are based on hard margin SVM, with the model tries to classify all the samples correctly, although it might generalize badly. Soft margin SVM allows some of the training samples to be misclassified with penalty of the outlier, thus generalizes well. The idea of soft margin SVM is we relax the constraints as much as necessary but punish the relaxation of a constraint. We introduce a slack variable $\xi_i \geq 0$ for every training sample $\mathbf{x}_i$, which gives the distance of how far the margin is violated by this training sample in units of $\|\mathbf{w}\|$. The relaxed constraints are

$$\begin{aligned} \mathbf{w}^T \mathbf{x}_i + b &\geq +1 - \xi_i \quad \text{for } y_i = +1 \\ \mathbf{w}^T \mathbf{x}_i + b &\leq -1 + \xi_i \quad \text{for } y_i = -1 \end{aligned}$$

again we can unify these two constraints with

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \quad \text{for all } i$$

We add a penalty term (here 1-norm penalty, we can also use 2-norm penalty) into the primal function of SVM, i.e. we try to minimize the primal with the consideration of constraint relaxation

$$f_0(\mathbf{w}, b, \xi) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i$$

with $C > 0$ denotes how heavy a violation is punished. For misclassified samples, the bigger the distance to the hyperplane, the larger the C is. $C \rightarrow \infty$ is then the hard margin SVM. We see that soft margin SVM doesn't change the position of the original hyperplane, but moves the additional two hyperplane up and down for a better generalization.

Soft margin SVM is still a constrained optimization problem with the primal function contained the penalty of violation

$$\begin{aligned} \text{minimize} \quad & f_0(\mathbf{w}, b, \xi) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i \\ \text{subject to} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i \geq 0 \\ & \xi_i \geq 0 \end{aligned}$$

The optimal solution of the slack variables ξ_i is

$$\xi_i = \begin{cases} 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) & \text{if } y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1 \\ 0 & \text{else} \end{cases}$$

we see that if $y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1$, we have $\xi_i = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) > 0$, means that the slack variable is not going to be zero if the points are not perfectly (i.e. points within the margin or cross the original hyperplane in the

middle) separated. If the points cross the middle hyperplane, they have a ξ_i bigger than 1. Plug in the optimal solution of ξ_i into the Lagrangian we get

$$\text{minimize}_{\mathbf{w}, b} \quad \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \max\{0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i)\}$$

and this is the hinge loss function that penalizes the points lie within the margin. In general, the hinge loss function has the form

$$E_{\text{hinge}}(z) = \max\{0, 1 - z\}$$

The zero zone of hinge loss function corresponds to those non support vectors, means that all these non support vectors are not part of the hyperplane decision process.

To solve this constrained optimization problem, we go through the recipe again

Step 1: write down Lagrangian

$$\begin{aligned} L(\mathbf{w}, b, \xi, \alpha, \mu) = & \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \mu_i \xi_i \\ & - \sum_{i=1}^N \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] \end{aligned}$$

Step 2: Minimize $L(\mathbf{w}, b, \xi, \alpha, \mu)$ w.r.t $\mathbf{w}$, b and ξ

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b, \xi, \alpha, \mu) = \mathbf{w} - \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i = 0$$

so we have the Lagrangian optimizer

$$\mathbf{w}^* = \sum_{i=1}^N \alpha_i y_i \mathbf{x}_i$$

same as the hard margin SVM, so we do see that soft margin SVM doesn't change the position of the original hyperplane.

Minimizing the Lagrangian w.r.t b we have

$$\nabla_b L(\mathbf{w}, b, \xi, \alpha, \mu) = \sum_{i=1}^N \alpha_i y_i = 0$$

same as hard margin SVM there's no close-form solution for the Lagrangian optimizer b^* , so again we add this term to the constraint.

Minimizing the Lagrangian w.r.t ξ_i (not ξ) we have

$$\nabla_{\xi_i} L(\mathbf{w}, b, \xi, \alpha, \mu) = C - \alpha_i - \mu_i = 0$$

again there's no close-form solution for the Lagrangian optimizer ξ_i . Because α_i and μ_i are Lagrangian multipliers, we have the dual feasibility $\alpha_i \geq 0$, $\mu_i \geq 0$, and from $\nabla_{\xi_i} L(\mathbf{w}, b, \xi, \alpha, \mu)$ we get $\alpha_i = C - \mu_i$, thus we get

$$0 \leq \alpha_i \leq C$$

which is still a linear constraint (box constraint). We also add this term to the constraint.

Substitute the Lagrangian optimizer $\mathbf{w}^*$ back to $L(\mathbf{w}, b, \xi, \alpha, \mu)$

$$\begin{aligned}
L(\mathbf{w}, b, \xi, \alpha, \mu) &= \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \mu_i \xi_i - \\
&\quad \left[\sum_{i=1}^N \alpha_i y_i (\mathbf{w}^T \mathbf{x}_i + b) - \sum_{i=1}^N \alpha_i + \sum_{i=1}^N \alpha_i \xi_i \right] \\
&= \frac{1}{2} \mathbf{w}^T \mathbf{w} + \underbrace{\sum_{i=1}^N (C - \mu_i - \alpha_i) \xi_i}_{=0} - \sum_{i=1}^N \alpha_i y_i (\mathbf{w}^T \mathbf{x}_i + b) + \sum_{i=1}^N \alpha_i \\
&= \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i y_i (\mathbf{w}^T \mathbf{x}_i + b) + \sum_{i=1}^N \alpha_i \\
&= -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j + \sum_{i=1}^N \alpha_i
\end{aligned}$$

we found that the result is exactly the same as the harg margin SVM. Maximizing it, we get the dual function same as the hard margin SVM but with different constraints

$$\begin{aligned}
&\text{maximize } g(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j \\
&\text{subject to } \sum_{i=1}^N \alpha_i y_i = 0 \\
&\quad 0 \leq \alpha_i \leq C
\end{aligned}$$

9 Kernels

9.1 Feature space

In many cases the data is not directly linearly separable in the origin space, we need to use the basis function to map the data into some high dimensional feature space, where the data can be linearly separable

$$\phi : \mathbb{R}^D \rightarrow \mathbb{R}^M \quad \mathbf{x}_i \mapsto \phi(\mathbf{x}_i)$$

9.2 Kernel trick

Kernel trick is the way how we avoid calculating complicated basis transformation. We try to find a feasible calculation in the space before the basis transformation to have the same results as the inner product in the space after the basis transformation. In a another word, any methods that have the same results as the inner product in the high-dimensional feature space, is a kernel.

Kernel trick can be used in any model that can be formulated such that it only depends on the inner products $\mathbf{x}_i^T \mathbf{x}_j$, e.g. linear regression, k-nearest neighbors etc.

The SVM without basis function has the following dual function

$$g(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j$$

For basis transformation this means that

$$g(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

Kernel function Define the kernel function as $k : \mathbb{R}^D \times \mathbb{R}^D \rightarrow \mathbb{R}$ and rewrite the dual as

$$g(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j)$$

A kernel is called valid if it corresponds to an inner product in some feature space. Or as said in Mercer's theorem, a kernel is valid if it gives rise to a positive semi-definite kernel matrix $\mathbf{K}$ for any input data $\mathbf{X}$. Kernel matrix $\mathbf{K} \in \mathbb{R}^{N \times N}$ is defined as

$$\mathbf{K} = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \cdots & k(\mathbf{x}_1, \mathbf{x}_N) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \cdots & k(\mathbf{x}_2, \mathbf{x}_N) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_N, \mathbf{x}_1) & k(\mathbf{x}_N, \mathbf{x}_2) & \cdots & k(\mathbf{x}_N, \mathbf{x}_N) \end{pmatrix}$$

If we happened to use a non-valid kernel, then the optimization problem might be non-convex, so we may not get a globally optimal solution.

Kernel preserving operations Let $k_1 : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ and $k_2 : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ be kernels, with $\mathcal{X} \subseteq \mathbb{R}^N$, the following functions k are kernels as well:

- $k(\mathbf{x}_1, \mathbf{x}_2) = k_1(\mathbf{x}_1, \mathbf{x}_2) + k_2(\mathbf{x}_1, \mathbf{x}_2)$
- $k(\mathbf{x}_1, \mathbf{x}_2) = c \cdot k_1(\mathbf{x}_1, \mathbf{x}_2)$, with $c > 0$
- $k(\mathbf{x}_1, \mathbf{x}_2) = k_1(\mathbf{x}_1, \mathbf{x}_2) \cdot k_2(\mathbf{x}_1, \mathbf{x}_2)$
- $k(\mathbf{x}_1, \mathbf{x}_2) = k_3(\phi(\mathbf{x}_1), \phi(\mathbf{x}_2))$, with the kernel k_3 on $\mathcal{X}' \subseteq \mathbb{R}^M$ and $\phi : \mathcal{X} \rightarrow \mathcal{X}'$
- $k(\mathbf{x}_1, \mathbf{x}_2) = \mathbf{x}_1^T \mathbf{A} \mathbf{x}_2$, with $\mathbf{A} \in \mathbb{R}^N \times \mathbb{R}^N$ symmetric and positive semi-definite

Example of kernels Following are kernels that we use very often

- Polynomial: $k(\mathbf{a}, \mathbf{b}) = (\mathbf{a}^T \mathbf{b})^p$ or $(\mathbf{a}^T \mathbf{b} + 1)^p$
- Gaussian kernel: maps into infinite-dimensional feature space

$$k(\mathbf{a}, \mathbf{b}) = \exp\left(-\frac{\|\mathbf{a} - \mathbf{b}\|^2}{2\sigma^2}\right)$$

- Sigmoid: $k(\mathbf{a}, \mathbf{b}) = \tanh(\kappa \mathbf{a}^T \mathbf{b} - \delta)$ for $\kappa, \delta > 0$

9.3 Kernelized SVM

We denote the set of support vectors (points $\mathbf{x}_i$ for which it holds $0 < \alpha_i < C$ and $\xi_i = 0$) as $\mathcal{S}$. From the complementary slackness condition, they must satisfy

$$y_i \left(\sum_{\{j|\mathbf{x}_j \in \mathcal{S}\}} \alpha_j y_j k(\mathbf{x}_i, \mathbf{x}_j) + b \right) = 1$$

and the bias can be recovered as

$$b = y_i - \left(\sum_{\{j|\mathbf{x}_j \in \mathcal{S}\}} \alpha_j y_j k(\mathbf{x}_i, \mathbf{x}_j) \right)$$

Thus, a new point $\mathbf{x}$ can be classified as

$$h(\mathbf{x}) = \text{sgn} \left(\sum_{i=1}^N \alpha_i y_i k(\mathbf{x}_i, \mathbf{x}) \right)$$

10 Deep Learning

10.1 Feed-Forward Neural Network

Feed-Forward Neural Network is also known as Multi-layered Perceptron (MLP), or Fully-connected Neural Network.

The reason that we use non-linear activation function: if we use pure linear operation, we don't need multiple layers at all

$$\begin{aligned} f(\mathbf{x}, \mathbf{W}) &= \mathbf{W}_k(\mathbf{W}_{k-1}(\dots(\mathbf{W}_0\mathbf{x})\dots)) \\ &= \mathbf{W}'\mathbf{x} \end{aligned}$$

for non-linear functions we usually can not simplify it

$$f(\mathbf{x}, \mathbf{W}) = \mathbf{W}_k \sigma_k(\mathbf{W}_{k-1} \sigma_{k-1}(\dots \sigma_0(\mathbf{W}_0\mathbf{x}) \dots))$$

Universal approximation theorem An MLP with a linear output layer and one hidden layer can approximate any continuous function defined over a closed and bounded subset $\mathbb{R}^D$, if the number of hidden units is large enough.

The reason that we use deep neural network is that if we use a few layers we would need a large number of hidden units (and therefore parameters). We can get the same representation power by adding more hidden layers, fewer hidden units, and fewer parameter. In that way, different high-level features share lower-level features.

10.2 Activation functions

- Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- tanh:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- ReLU:

$$\max(0, x)$$

- Leaky ReLU:

$$\max(0.1x, x)$$

Sigmoid and the tanh can saturate if the input is too small or too big, i.e. they can cause vanishing gradient problem. ReLU alleviates the above problem because the gradient is always 1 at least on positive input. However, ReLU can cause dead ReLU unit when the input is negative due to e.g. a large negative bias, in this case the gradient w.r.t the weights becomes zero and the unit will remain at this state forever. Leaky ReLU alleviates the dead ReLU problem.

We use non-linear activation function for classification tasks and identity or no activation function for regression tasks.

10.3 Choice of loss and last layer output

The loss function and the choice of activation function of the last layer depend on the task we do.

Example: Binary classification We use binary cross-entropy loss and sigmoid function as the activation function of the last layer

Output:

$$y = \sigma(a) = \frac{1}{\exp(-a)} = f(\mathbf{x}, \mathbf{w})$$

Loss function:

$$E(\mathbf{w}) = - \sum_{i=1}^N (y_i \log f(\mathbf{x}_i, \mathbf{W}) + (1 - y_i) \log[1 - f(\mathbf{x}_i, \mathbf{W})])$$

Example: Standard multi-class classification We use cross-entropy loss and softmax function as the activation function of the last layer
Output:

$$y_k = \frac{\exp(a_k)}{\sum_j \exp(a_j)} = f_k(\mathbf{x}, \mathbf{W})$$

Loss function:

$$E(\mathbf{W}) = - \sum_{n=1}^N \sum_{k=1}^K (y_{nk} \log f_k(\mathbf{x}_n, \mathbf{W}))$$

10.4 Parameter Learning

In practice $E(\mathbf{W})$ is often non-convex, so we use gradient descent to compute a solution which is good enough for the praxis.

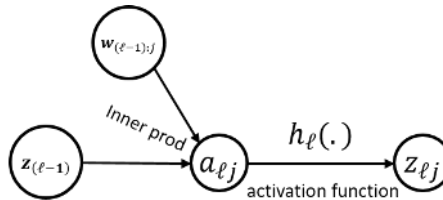
$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \alpha \nabla_{\mathbf{W}} E(\mathbf{W}^{(t)})$$

Ways to compute gradient:

- By hand: tricky and cumbersome
- Numeric: has to be done for each parameter independently, too many operations
- Symbolic differentiation: explicitly writing down the gradient function for each parameter is very expensive
- Automatic differentiation: e.g. backpropagation, automatically and efficiently

Backpropagation

- Forward pass: write down the value above the edges in the computational graph
- Backward pass: write down the gradient under the edges in the computational graph. gradient = local gradient $\times$ upstream gradient



In the above figure, we denote the following parameters per layer as

- w_{lij} : weight at layer l , input node i and output node j
- $z_{li}^{(n)}$: the value of a neuron at layer l , node index i and instance n
- $a_{li}^{(n)}$: the value of a logit at layer l , node index i and instance n

$$\mathbf{a}_l^{(n)} = \mathbf{W}_{l-1} \cdot \mathbf{z}_{l-1}^{(n)}$$

- $h_l(\cdot)$: the activation function of the l -th layer

$$z_{lj}^{(n)} = h_l(a_{lj}^{(n)})$$

Apply chain rule we have the gradient computation

$$\frac{\partial E_n}{\partial w_{(l-1)ij}} = \underbrace{\frac{\partial E_n}{\partial a_{lj}^{(n)}}}_{\delta_{lj}^{(n)}} \underbrace{\frac{\partial a_{lj}^{(n)}}{\partial w_{(l-1)ij}}}_{z_{(l-1)i}^{(n)}}$$

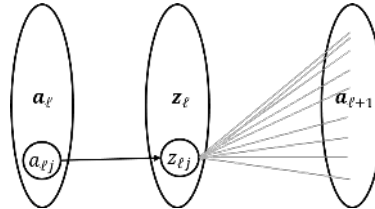
with $\delta_{lj}^{(n)}$ called the error of j -th neuron at the l -th layer, the upstreaming gradient and the second term the local gradient.

Forward pass we just evaluate the function

$$\mathbf{a}_l^{(n)} = \mathbf{W}_{l-1} \mathbf{z}_{l-1}^{(n)}$$

$$\mathbf{z}_l^{(n)} = \mathbf{h}_l(\mathbf{a}_l^{(n)})$$

Backward pass we compute the upstreaming and local gradient and then multiply them together. For upstreaming gradient (i.e. the error) we have



$$\frac{\partial E_n}{\partial a_{lj}^{(n)}} = \frac{\partial E_n}{\partial a_{(l+1)j}^{(n)}} \frac{\partial a_{(l+1)j}^{(n)}}{\partial z_{lj}^{(n)}} \frac{\partial z_{lj}^{(n)}}{\partial a_{lj}^{(n)}}$$

$$\Rightarrow \delta_{lj}^{(n)} = (\delta_{l+1}^{(n)})^T \mathbf{W}_{lj} : h'_l(a_{lj}^{(n)})$$

the derivative of activation function is always known.

We should always keep in mind that the vanishing or exploding gradient problem caused by activation functions or by repetition of a parameter (i.e. multiply very small or big numbers many times, depends on how deep the network is).

10.5 CNN

Convolution operation Convolution operation averages the information within the convolutional window, thus reduces the dimensionality of the input space.

If we want to keep the input dimensionality as constant, we should do zero padding around the image. Different convolutional kernels extract different features from the image. In the lower layers, the network tries to catch lower-level features of the image like edges, corners etc, the more abstract higher-level features are captured in deep layers.

Compute the output size after the convolution operation. Denote number of padding and stride as p and s , we have

$$\begin{cases} h_{out} = (h_{in} - h_{kernel} + 2 \cdot p) / s + 1 \\ w_{out} = (w_{in} - w_{kernel} + 2 \cdot p) / s + 1 \end{cases}$$

The output after the convolution operation is called feature map.

Maxpooling operation Convolution operation was not originally designed for dimensionality reduction of the input image. It was actually the Maxpooling operation which was designed to perform reducing dimensionalities. The 2x2 Maxpooling reduces the size of the feature map to its half.

10.6 RNN

Recurrent neural network expands the time, deals with sequence structured information, while recursive neural network (sometimes also abbreviated as RNN) expands the space and deals with hierarchical structured information. Don't mess up these two. For recurrent neural network, the output depends not only on the input but also on the history information.

The general RNN still suffers from vanishing or exploding gradient problem. A very popular model which aims to alleviate this issue is the Long-Short Term Memory Network (LSTM).

10.7 Training deep neural network

Weight initialization If two hidden units have exactly the same bias and incoming and outgoing weights, they will always get the same gradient, thus they never learn different features. So we need to break the symmetry by initializing with small random values.

Regularization To prevent overfitting, we need to add penalty to the weights. Typically we use L_2 norm penalty. Sometimes we also use L_1 norm penalty to promote some sparsity. Other regularization methods are dataset augmentation (e.g. rotate/translate/skew/change lighting of images), injecting noise, parameter tying and sharing, and the commonly used dropout.

Dropout: everytime a learning sample is learned, we randomly put 0 to each hidden unit with probability of 0.5, i.e. we randomly disable some neuron by setting their outputs to zero. This is actually sampling from 2^H different network architectures which share the same weights.

Hyperparameter tuning There are a lot of hyperparameter which need to be finetuned. We can use the hyperparameter optimization (like random search or Bayesian optimization etc.) to find a good set of hyperparameters. We can also learn the hyperparameter with gradient descent, i.e. compute the gradient of the loss w.r.t hyperparameters. This is called META-learning, i.e. the network learns how to learn.

Deep learning frameworks

- TensorFlow
- PyTorch
- MXNet
- Caffe2
- ...

Every deep learning framework has its own definition of computation graphs. PyTorch for instance, has dynamic computation graph which allows changing architecture. TensorFlow on the other hand has static computation graph.

Tricks for training neural networks

- Use only differentiable operations
- Always try to overfit the model on a small batch of the training set to make sure that the model is right
- Start with small models and gradually add complexity while monitoring how the performance improves
- Be aware of the properties of activation functions, e.g. no sigmoid output when doing regression
- Monitor the training procedure and use early stopping

11 PCA

Find a coordinate system in which the data are linearly uncorrelated (Feature selection is a linear transformation). The dimensions with no or low variance can be then ignored since they don't carry much information. The motivation of PCA is that the data often lies on a low dimensional subspace, which means the data is just a low dimensional object in the high dimensional space (like a line in a plane or a plane in 3D space).

11.1 Determin the principle component

The goal of PCA is that we transform the data, such that the covariance between the new dimension is 0 (means that the new dimensions are linearly uncorrelated). Given N d-dimensional data points $\{x_i\}_{i=1}^N, x+i \in \mathbb{R}^d, \forall i \in \{1, \dots, N\}$, represent the data points by a matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, i.e. each row is a data point and each column is one dimension of the data points $\mathbf{X}_j, j \in \{1, \dots, d\}$

$$\mathbf{X} = \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{N1} & \cdots & x_{Nd} \end{bmatrix}$$

The general approach of PCA:

- Center the data by shifting it to the mean

$$\tilde{x}_i = x_i - \bar{x}$$

$$\mathbf{X} \rightarrow \tilde{\mathbf{X}}$$

- Compute the covariance matrix of the data $\mathbf{X}$ by determining the variances $\text{var}(X_j), j \in \{1, \dots, d\}$ for each dimension. The covariance $\text{cov}(X_{j_1}, X_{j_2})$ between two distinct dimensions $j_1, j_2, \forall j_1 \neq j_2 \in \{1, \dots, d\}$

$$\text{var}(X_j) = \frac{1}{N} \sum_{i=1}^N (x_{ij} - \bar{x}_j)^2 = \frac{1}{N} \cdot X_j^T X_j - \bar{x}_j^2$$

$$\begin{aligned} \text{cov}(X_{j_1}, X_{j_2}) &= \frac{1}{N} \sum_{i=1}^N (x_{ij_1} - \bar{x}_{j_1}) \cdot (x_{ij_2} - \bar{x}_{j_2}) \\ &= \frac{1}{N} \cdot X_{j_1}^T X_{j_2} - \bar{x}_{j_1} \bar{x}_{j_2} \end{aligned}$$

The covariance matrix $\Sigma_{\mathbf{X}}$ is therefore

$$\Sigma_{\mathbf{X}} = \begin{bmatrix} \text{var}(X_1) & \text{cov}(X_1, X_2) & \cdots & \text{cov}(X_1, X_d) \\ \text{cov}(X_2, X_1) & \text{var}(X_2) & \cdots & \text{cov}(X_2, X_d) \\ \vdots & \cdots & \ddots & \vdots \\ \text{cov}(X_d, X_1) & \text{cov}(X_d, X_2) & \cdots & \text{var}(X_d) \end{bmatrix}$$

recall that the covariance of a variable itself is the variance. We see that the covariance matrix is symmetric and square.

- Do Eigenvector decomposition to the covariance matrix to transform the coordinate system

$$\mathbf{X} \rightarrow \text{cov}(\mathbf{X}) \rightarrow \mathbf{V} \cdot \mathbf{W} \cdot \mathbf{V}^T$$

where $\mathbf{V}$ is the matrix of eigenvectors and $\mathbf{W}$ is the diagonal matrix of eigenvalues.

$$\Sigma_{\tilde{\mathbf{X}}} = \Gamma \cdot \Lambda \cdot \Gamma^T$$

where $\Gamma \in \mathbb{R}^{d \times d}$ with columns being normalized eigenvectors γ_i and Λ an diagonal matrix with eigenvalues λ_i as the diagonal elements. Λ is also the covariance matrix of the new coordinate system after the transformation. Noticed that the Eigenvector decomposition works on symmetric matrices.

11.2 Dimension reduction with PCA

From the spectral theorem, we know that the eigenvectors of a symmetric matrix form an orthogonal basis, and from the goal of PCA, we want an optimal orthogonal transformation that reduces the correlations of the data. After eigendecomposition, the new coordinate system is $Y = \tilde{\mathbf{X}} \cdot \Gamma$, i.e. the centered data multiplied with the eigenvector diagonal matrix of the eigendecomposition. Thus, we can do a truncation of Γ by keeping only columns of Γ corresponding to the largest k eigenvalues $\lambda_1, \dots, \lambda_k$, i.e.

$$Y_{\text{reduced}} = \tilde{\mathbf{X}} \cdot \Gamma_{\text{truncated}}$$

For picking k we use the 90% rule, the k eigenvector corresponding to the k largest eigenvalues should explain 90% of the energy

$$\sum_{i=1}^k \lambda_i \geq 0.9 \cdot \sum_{i=1}^d \lambda_i$$

which is very expensive since we need to compute all eigenvalues in order to have k . To improve the efficiency, we can use power iteration (a.k.a Von Mises iteration) to get the k . In general, let X be a matrix and v be an arbitrary vector, power iteration has following steps:

Step 1: Do eigenvector decomposition to X

$$X = \Gamma \cdot \Lambda \cdot \Gamma^T = \sum_{i=1}^d \lambda_i \cdot \gamma_i \cdot \gamma_i^T$$

Step 2: Define deflated matrix

$$\hat{X} = X - \lambda_1 \cdot \gamma_1 \cdot \gamma_1^T$$

where we subtract the largest eigenvector from X . In PCA, the X will be the covariance matrix $\Sigma_{\hat{X}}$.

Step 3: Apply the power iteration. Let v be an arbitrary vector, iteratively compute

$$v \leftarrow \frac{\hat{X} \cdot v}{\|\hat{X} \cdot v\|}$$

in each step v is simply multiplied with $\hat{X}$ and normalized. When v converges, v corresponds to the eigenvector of $\hat{X}$ with second largest absolute value.

Step 4: Check whether the 90% rule is fulfilled. If not, follow step 1 to 3 to compute the eigenvector that corresponds to the third largest eigenvalue. Repeat until the 90% rule is fulfilled.

11.3 Alternative views of PCA

Maximum variance formulation Project the data to a lower dimensional space $\mathbb{R}^k$, $k \ll d$ while maximizing the variance of the projected data.

Example: we project the data into 1D space, i.e. $k = 1$. We know that the projection is done by multiplying a 1D unit vector to the data, let's say u_1 . Since we want to maximize the variance after the projection, we shall compute the variance and see what happens. The variance can be computed as

$$\begin{aligned} \text{var}(X_{\text{projected}}) &= \frac{1}{N} \sum_{i=1}^N \left(\underbrace{u_1^T x_i}_{\text{proj. data}} - \underbrace{u_1^T \bar{x}}_{\text{proj. mean}} \right)^2 \\ &= u_1^T S u_1 \end{aligned}$$

where S is the sample covariance matrix

$$S = \begin{bmatrix} \text{var}(x_1) & \text{cov}(x_1, x_2) & \cdots & \text{cov}(x_1, x_N) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(x_N, x_1) & \text{cov}(x_N, x_2) & \cdots & \text{var}(x_N) \end{bmatrix}$$

Since u_1 is a unit vector, this becomes a constrained optimization problem

$$\begin{aligned} &\text{maximize} && u_1^T S u_1 \\ &\text{subject to} && u_1^T u_1 = 1 \end{aligned}$$

we can then write down the Lagrangian as

$$L = u_1^T S u_1 + \lambda_1 (1 - u_1^T u_1)$$

Solving this constrained optimization problem we get the stationary point of the Lagrangian $S u_1 = \lambda_1 u_1$. Multiply u_1^T from the left we get $u_1^T S u_1 = \lambda_1$.

Minimum error formulation Find an orthogonal set of k linear basis functions $w_j \in \mathbb{R}^d$ and corresponding low-dimensional projections $z_j \in \mathbb{R}^k$ such that the average reconstruction error

$$J = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2$$

is minimized with $\hat{x}_i = W z_i + \mu$. In another word, find one low dimensional projection which allows us to reconstruct the original data with the most information being preserved.

11.4 PPCA

The drawback of PCA is that PCA can't deal with missing data, e.g. if a data point loses a dimension, we can't compute the covariance matrix anymore since it operates over all values in each dimension. PPCA on the other hand, handles perfectly the missing data issue by introducing a latent variable z and transforming its distribution into an observable space. We assume that the data points x_i are generated by this latent variable z which obeys standard Gaussian distribution $z_i \sim N(0, I)$. The generation of the data point x_i is therefore

$$x_i = W z_i + \mu + \epsilon_i$$

where μ is the shift term (the mean) and $\epsilon_i \sim N(0, \Phi)$ the projection uncertainty (error) with same variance $\Phi_i = \sigma^2$. We see this is very similar to linear regression $y = w^T x_i + \epsilon$ where $\epsilon \sim N(0, \sigma^2)$. We can see through the generation process that x_i given z_i is also a Gaussian distribution

$$p(x_i|z_i) \sim N(Wz_i + \mu, \sigma^2 I)$$

In order to get the distribution of x_i we integrate the latent variable z_i out. From Bayesian equation we get

$$p(x_i) = \int p(x_i|z_i)p(z_i)dz$$

thus, $x_i \sim N(\mu, WW^T + \sigma^2 I)$, i.e. we can now compute the probability of each single data point x_i . To compute the probability of the entire dataset $\mathbf{X}$, a.k.a the likelihood function, and since each individual data point is independent, we get

$$p(\mathbf{X}) = \prod_{i=1}^N p(x_i) = \prod_{i=1}^N N(x_i|\mu, WW^T + \sigma^2 I)$$

Now we see how PPCA deals with the missing data: integrate the missing dimension out.

Example Assume a 3-dimensional data point $x = (x_1, x_2, x_3)^T$, and due to some reasons the second dimension is missing, i.e. $x = (x_1, \cdot, x_3)$. To compute $p(x_1, \cdot, x_3)$, we can do

$$p(x_1, \cdot, x_3) = \int p(x_1, x_3|x_2)p(x_2)dx_2$$

so while computing $p(\mathbf{X})$, if a data point has missing dimension, just use the integrated version of $p(x_i)$ for this data point.

We can take log-likelihood for the PPCA

$$LL = -\frac{N}{2}(d \ln(2\pi) + \ln|C| + \text{tr}(C^{-1}S))$$

where $C = WW^T + \sigma^2 I$ and $S = \frac{1}{N}(\sum_{i=1}^N (x_i - \mu)(x_i - \mu)^T)$. We then optimize w.r.t μ , W and σ^2 using MLE. The close form solution for W and σ is

$$W_{ML} = U_k(\Lambda_k - \sigma^2 I)^{\frac{1}{2}} V$$

where $U_k \in \mathbb{R}^{d \times k}$ the principle eigenvectors of S , $\Lambda_k \in \mathbb{R}^{k \times k}$ the diagonal matrix of corresponding eigenvalues of S and $V \in \mathbb{R}^{k \times k}$ an arbitrary rotation matrix (can be set to $I_{k \times k}$). Notice that if we choose $\sigma \rightarrow 0$, $W_{ML} = U_k(\Lambda_k)^{\frac{1}{2}} V$, i.e. PPCA is then PCA.

The closed form solution of σ^2 is then

$$\sigma_{ML}^2 = \frac{1}{d-k} \sum_{j=k+1}^d \lambda_j$$

where λ_j are the corresponding eigenvalues. This is the variance we lost while doing up-projection, thus dimension $j+1$ to k don't carry much information.

The advantages of PPCA are

- Optimizing the model is done through optimizing the log-likelihood function using gradient descent, thus there's no need to compute the covariance matrix since the covariance matrix could be very large if the data is high dimensional
- PPCA is generative model, thus capable of generating new data
- Easy to combine multiple PCA models into a mixture of PCA, i.e. we can use multiple PCA models to fit the data
- PPCA can handle the missing data by integrating the missing dimension out for each $p(x_i)$

12 SVD

Singular value decomposition is a method of low rank approximation. The goal of SVD is to find the best low rank approximation by minimizing the reconstruction error. Given matrix $A \in \mathbb{R}^{n \times d}$ and matrix $B \in \mathbb{R}^{n \times d}$ with A and B have the same shape but different ranks, we want to minimize the reconstruction error

$$\|A - B\|_F^2 = \sum_{i=1}^N \sum_{j=1}^D (a_{ij} - b_{ij})^2$$

we shall see how to minimize this in a minute.

12.1 Definition

Each real matrix $A \in \mathbb{R}^{n \times d}$ (doesn't matter whether it's symmetric or not), can be decomposed into

$$A = U \cdot \Sigma \cdot V^T = \sum_{i=1}^r \sigma_i \cdot \underbrace{u_i \circ v_i^T}_{\text{rank}=1}$$

where $U \in \mathbb{R}^{n \times r}$, $\Sigma \in \mathbb{R}^{r \times r}$, $V \in \mathbb{R}^{d \times r}$. U and V are column orthogonal ($U^T U = V^T V = I$) and are called as left singular vector and right singular vector. Σ is a diagonal matrix with entries called singular values sorted in decreasing order ($\sigma_1 \geq \sigma_2 \geq \dots \geq 0$). r is the rank of A , i.e. $\text{rank}(A) = r$.

SVD is (almost) unique, since multiply the corresponding entries in U and V with -1 doesn't change the result. Besides that, SVD has very good interpretability. In practice we always define the interpretability after SVD with the decomposition which has the most positive entries in the singular vectors.

12.2 Best approximation

The best approximation with SVD is to set the smallest singular value to zero (proof see piazza). This is the so called truncated SVD.

Since $A = U \Sigma V^T$, the projection of original data is therefore $A_{proj} = U \Sigma$, or $A_{proj} = A V$ since V is orthogonal and if we multiply V from the right side, we get $U \Sigma = A V$. And because SVD is sum of rank one matrices weighted by singular value, same as PCA, we can use 90% rule to decide how many singular values we should pick for the reconstruction

$$\sum_{i=1}^k \sigma_i^2 \geq 0.9 \sum_{i=1}^r \sigma_i^2$$

where r is still the rank of A .

12.3 SVD and PCA: Comparison

Given centered data X , SVD does the following

- $X = U \Sigma V^T$
- Projected data obtained by $X \cdot V$ or $U \cdot \Sigma$. In the truncated SVD, we prefer the projection $X \cdot V$ rather than $U \cdot \Sigma$ since for $X \cdot V$ we only need to compute the top k singular vectors while $U \cdot \Sigma$ need all singular values.

while PCA does following

- Covariance matrix: $X^T X$
- Eigendecomposition: $X^T X = \Gamma \Lambda \Gamma^T$
- Projected data obtained by $X \cdot \Gamma$

If we plug in SVD of X into PCA, we get

$$\begin{aligned} X^T X &= (U \Sigma V^T)^T U \Sigma V^T = V \Sigma^T \underbrace{U^T U}_{=I, U \text{ orth.}} \Sigma V^T \\ &= V \underbrace{\Sigma^T \Sigma}_{=\Sigma^2, \Sigma \text{ sym.}} V^T = V \Sigma^2 V^T \end{aligned}$$

so $\Gamma = V$, $\Sigma^2 = \Lambda$, we see that PCA and SVD are equivalent. Thus, transform the data such that dimensions of new space are uncorrelated and discard new dimensions with smallest variance, is equal to, find the optimal low-rank approximation by minimizing the Frobenius norm of reconstruction error.

13 Matrix Factorization

13.1 Latent Factor Model

Matrix factorization is very often used in recommendation system. The idea is that for a given utility matrix R (or called rating matrix, set of tuples (i, x) of user x rates item i with a rating of r_{xi}), we can reveal some characteristics of users and items by decomposing the utility matrix into user-factor and factor-item matrix, so we can obtain user's preferences and characteristics of items, and also, we explicitly obtain the rank of the utility matrix.

To evaluate the model, we use RMSE, the root mean square error

$$RMSE = \frac{1}{|R|} \sqrt{\sum_{(i,x) \in R} (\hat{r}_{xi} - r_{xi})^2}$$

with $\hat{r}_{xi}$ the predicted rating and r_{xi} the true rating. Notice that the RMSE is just SSE (standard square error) normalized by the number of rating. In recommendation system, a low RMSE means good recommendation, since the system provides similar rating as the user's preference.

Recall that in SVD, we decompose the data matrix and then use the projection to represent the reconstruction, we can do the same to the utility matrix as in

$$R \approx Q \cdot P^T$$

with $Q = U\Sigma$. By doing that, we split the rating matrix $R \in \mathbb{R}^{n \times d}$ into user-factor matrix $Q \in \mathbb{R}^{n \times k}$ and factor-item matrix $P \in \mathbb{R}^{d \times k}$ (remember k is the rank of R). Because the natural sparsity of the rating matrix, the traditional matrix decomposition techniques can sometimes be very counter intuitive if we decompose the sparse matrix into dense matrices, and also the complexity of decomposition may be very high. So the common way of dealing with it is to use the existing rating to compute the prediction error, then optimize the prediction error using gradient descent techniques.

We also see that in the user-factor matrix Q , each row q_x represents how much does the user likes about the latent factors of the item, i.e. the column dimension represents the rank of the rating matrix. In the factor-item matrix P^T , each column p_i^T represents how much does the item belongs to the latent factors.

Notice that SVD computes the error term over all entries in R , and the sparsity of R means that there are entries missing (just from the fact that a user can't rate all the items). But SVD treats missing entries as zero-rating, so this is critical for many applications. So we have to modify the prediction error by only summing over the existing entries

$$\min_{P, Q} \sum_{(i,x) \in R} (r_{xi} - q_x \cdot p_i^T)^2$$

here we don't require columns of P and Q to be orthogonal or unit length since this is not even true SVD, because SVD always sums over all entries. Notice that although the modified prediction error has a quadratic form, this is not a convex function since it has two variables.

13.2 Alternating Optimization

In order to optimize a function of two variables, we use alternating optimization (a.k.a block coordinate minimization) technique to optimize the prediction error by split the original optimization problem into sequence of simple OLS problems. First we pick initial values for P and Q , a good way to initialize P and Q is just to use SVD of R (missing entries are replaced by 0). Then we alternatively keep one variable fix and optimize for the other. The whole process repeats until convergence. Write as pseudo code we have

- Initialize $P^{(0)}$ and $Q^{(0)}$, $t = 0$
- $P^{(t+1)} = \arg \min_P f(P, Q^{(t)})$
- $Q^{(t+1)} = \arg \min_Q f(P^{(t+1)}, Q)$
- $t = t + 1$
- Repeat until convergence

We now take a deeper look at the optimization step $P^{(t+1)} = \arg \min_P f(P, Q^{(t)})$. Since we fix Q to optimize P , we can now optimize each vector p_i independently

$$\min_P \sum_{(i,x) \in R} (r_{xi} - q_x \cdot p_i^T)^2 = \sum_{i=1, \dots, d} \min_{p_i} \sum_{x \in R_{*,i}} (r_{xi} - q_x \cdot p_i^T)^2$$

here $R_{*,i} = \{x | (i, x) \in R\}$ means all users that have rated item i . Equivalently if we fix P to optimize Q , we can also optimize each vector q_x independently

$$\min_Q \sum_{(i,x) \in R} (r_{xi} - q_x \cdot p_i^T)^2 = \sum_{x=1, \dots, n} \min_{q_x} \sum_{i \in R_{*,x}} (r_{xi} - q_x \cdot p_i^T)^2$$

with $R_{*,x} = \{i | (i, x) \in R\}$ means all items that have been rated by users.

Further more, we see that $\min_{p_i} \sum_{x \in R_{*,i}} (r_{xi} - q_x \cdot p_i^T)^2$ is an ordinary least square regression problem, which has the standard form

$$\min_w \sum_{i=1}^N (y_i - w^T x_i)^2$$

with optimal solution $w^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$. So we can obtain the optimal solution for p_i^T and q_x as well. For p_i^T it's just

$$p_i^T = \left(\frac{1}{|R_{*,i}|} \sum_{x \in R_{*,i}} q_x^T q_x \right)^{-1} \cdot \frac{1}{|R_{*,i}|} \sum_{x \in R_{*,i}} q_x^T r_{xi}$$

Since we use alternating optimization technique, there are drawbacks which come along with the method

- Solution is only an approximation
- No guarantee that is close to the optimal solution (local minima, saddle position etc.)
- Highly depends on initial solution, i.e. how we initialize P and Q

13.3 Rating Prediction

After learning Q and P , the next step is to estimate the missing rating of user x for item i , i.e. we want to fill up the blanks the rating matrix R . Since we just decompose the sparse rating matrix into dense matrices Q and P , we can now reconstruct the missing entries (i.e. the blank rating) in R by multiplying the corresponding row and column of R .

The challenge is that since we want our recommendation system to finally perform well on the test data, i.e. the data which never used during the training. So we want our prediction to be as close as possible to the original data, means that the projection of data should reconstruct the data as good as possible, so we want a large number of factors to represent the rating information. The thing is if the number of factors becomes too large, we will have overfitting issues, because in the alternating optimization process we have too many iterations but relative few data points, so we have more parameters than equations, thus the system is underdetermined. In order to prevent overfitting, we must add regularization term to the P and Q

$$\min_{P,Q} \underbrace{\sum_{\text{training}} (r_{xi} - q_i p_x^T)^2}_{\text{recons. error}} + \underbrace{\left[\lambda_1 \sum_x \|p_x\|^2 + \lambda_2 \sum_i \|q_i\|^2 \right]}_{\text{length}}$$

here λ_1 and λ_2 are user defined regularization parameters. We can still use alternating optimization to solve this regularized problem. By fix one variable, we see this is just the ridge regression problem. By adding the regularization terms, we force the parameter of P and Q to be smaller, because large values means that the model tries to fit the noise.

13.4 L2 vs. L1 Regularization

L2 regularization

- L2 tries to shrink the parameter vector equally.
- Large values are highly penalized due to the square in the L2 norm
- It's unlikely that any component will be exactly 0

L1 regularization

- L1 allows large values of parameter vector to shrink to 0

- L1 is suited to enforce sparsity of the parameter vector

The reason we prefer a sparse P and Q is that, first of all, it's not intuitive that our sparse rating matrix is decomposed into two dense matrices, this leads to better interpretation of the decomposition process. Secondly, sparse matrices have low requirement of storage, thus the computation can be also faster.

Matrix factorization is extremely powerful in

- Dimensionality reduction
- Data analysis/data understanding
- Prediction of missing values etc.

13.5 Further Factorization Models

Data is often given in form of non-negative values, e.g. rating values between 0 to 5, income, age... etc. But SVD might lead to factors containing negative values, which is not intuitive (it doesn't make sense that non-negative data is generated based on negative factors). The solution to this issue is to use non-negative matrix factorization

Non-Negative Matrix Factorization Given $A \in \mathbb{R}^{n \times d}$, $A \geq 0$ and integer k , find $P \in \mathbb{R}^{n \times k}$ and $Q \in \mathbb{R}^{k \times d}$ such that $\|A - P \cdot Q\|_F$ is minimized subject to $P \geq 0$ and $Q \geq 0$, i.e.

$$\begin{aligned} & \text{minimize} \quad \|A - P \cdot Q\|_F \\ & \text{subject to} \quad P \geq 0, Q \geq 0 \end{aligned}$$

we see that this is just constrained optimization problem. The easiest way to solve the problem is to use projected gradient descent.

Also sometimes the data can be in binary form, e.g. indicator matrices, with 1 indicates user buys the stuff and 0 don't. In this case we can apply the Boolean algebra (+ becomes "or" and $\cdot$ becomes "and") into the matrix factorization process.

Boolean Matrix Factorization Given Boolean matrix $A \in \{0, 1\}^{n \times d}$ and integer k , factorize A in Boolean matrix $B \in \{0, 1\}^{n \times k}$ and $C \in \{0, 1\}^{k \times d}$, i.e. $A \approx B \circ C$, such that $|A - B \circ C|$ is minimized.

13.6 Autoencoder

From dimensionality reduction techniques like PCA and SVD we know that the actual data usually lies on low dimensional space, which the data can be described as linear hyperplane in that space. But if the inner structure of data is non-linear, i.e. the data lies on a non-linear low dimensional manifold, there's nothing PCA and SVD can do to capture the non-linear structure of data. Unfortunately, we don't know whether the data has a non-linear structure, but if it does, we shall see there's no big change in the eigenvalues after doing SVD. Since PCA and SVD are both linear projection, we need to find a non-linear projection of the data.

An autoencoder is a neural network that finds a compact representation of the data by learning to reconstruct its input, i.e. $f_{dec}(f_{enc}(x)) \approx x$. Just like matrix factorization, we can optimize the reconstruction error

$$\min_W \frac{1}{N} \sum_{i=1}^N \|f(x_i, W) - x_i\|^2$$

If we use linear activation functions in the encoder and decoder network, we will see that this is again just lower rank approximation of the data like PCA and SVD

$$\begin{aligned} f_{enc}(x, W_1) &= xW_1 = z, & W_1 &\in \mathbb{R}^{D \times L} \\ f_{dec}(z, W_2) &= zW_2, & W_2 &\in \mathbb{R}^{L \times D} \\ \Rightarrow f_{dec}(f_{enc}(x)) &= xW_1W_2 \end{aligned}$$

the reconstruction error becomes

$$\begin{aligned}
& \min_{W_1, W_2} \frac{1}{N} \sum_{i=1}^N \|f(x_i, W) - x_i\|^2 \\
&= \min_{W_1, W_2} \frac{1}{N} \sum_{i=1}^N \|x_i W_1 W_2 - x_i\|^2 \\
&= \min_W \frac{1}{N} \sum_{i=1}^N \|x_i W - x_i\|^2
\end{aligned}$$

The encoder network projects the data to a lower dimension $f_{enc}(x) = z$, and the decoder network reconstructs the data from the latent representation z . The learning process will make the latent representation z compact. The latent representation z is therefore a very informative feature which captures important patterns of the input data. An autoencoder whose latent representation dimension is less than the input dimension $L \ll D$ is called undercomplete, while $L \geq D$ is called overcomplete.

To train the autoencoder, we add regularization term as usual to prevent overfitting. We also make the encoder and decoder networks sharing the weights.